

100 MuleSoft–AWS Integration Problems

Volume 5 — Advanced & Enterprise Patterns. 20 fully worked, real-project problems: multi-service orchestration, analytics and content delivery, distributed-transaction and streaming patterns, and the operational discipline — failover, cost control, deployment safety — a platform-grade MuleSoft–AWS integration needs.

Problems 81–100

Anypoint Studio

Postman

MuleSoft Developer Edition

Volume 5 — Advanced & Enterprise Patterns

The first four volumes worked through AWS one service family at a time. Volume 5 closes the series by combining them: orchestrating several AWS services from a single business process, reaching into analytics and content-delivery services this series hasn't touched yet, and applying the enterprise-grade patterns — distributed transactions, multi-region failover, cost-aware design, deployment safety — that separate a well-built integration from a platform a whole organization can depend on. It closes with a capstone that ties the entire series together into one integration platform.

SECTION 1 Multi-Service Orchestration & Enterprise Services (Problems 81–90)

#81

Orchestrating a Multi-Step Order-Fulfillment Flow Across S3, SQS, Lambda, and DynamoDB

AWS

PROBLEM STATEMENT

Fulfilling an order requires archiving the order document to S3, invoking a Lambda function to reserve inventory, recording the fulfillment state in DynamoDB, and notifying a downstream warehouse system via SQS — four different AWS services that all have to succeed, in the right order, for the order to be considered fulfilled.

DETAILED EXPLANATION

Rather than four independent calls that might partially succeed and leave an inconsistent state, this flow treats fulfillment as one orchestrated sequence: each step's output feeds the next, the DynamoDB state record is only written once inventory reservation has actually succeeded, and the SQS notification — the one step whose failure shouldn't block the order — fires last and outside the critical error path. This is the composition pattern that turns four separate AWS integrations, each covered individually in earlier volumes, into a single coherent business process.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

AWS AWS S3 Connector – Upload

AWS AWS Lambda Connector – Invoke

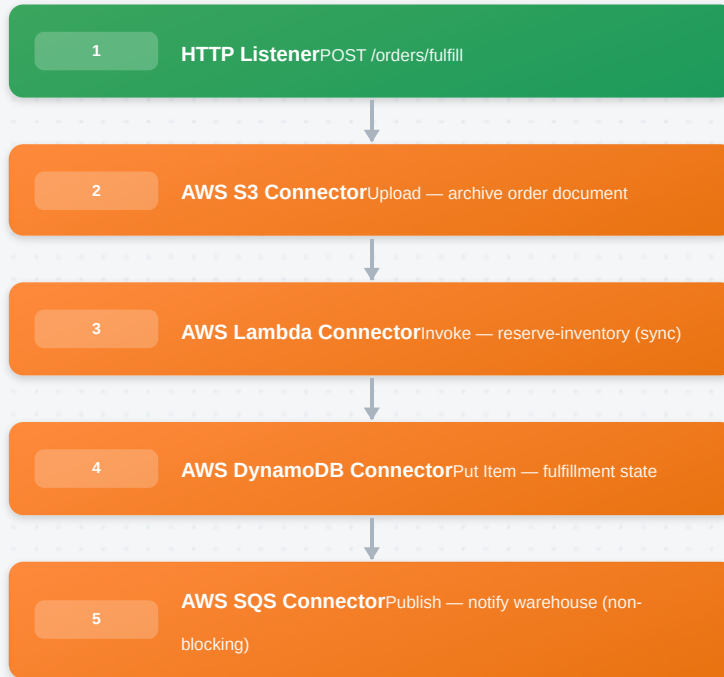
AWS AWS DynamoDB Connector – Put Item

AWS AWS SQS Connector – Publish

STEPS TO CREATE THE MULE FLOW

- 1 Archive the order document to S3 and invoke the inventory-reservation Lambda.
- 2 On successful reservation, write the fulfillment state record to DynamoDB.
- 3 Publish a non-blocking notification to the warehouse SQS queue.

FULFILL-ORDER-FLOW



MULE CONFIGURATION XML

```

<flow name="fulfill-order-flow">
  <http:listener path="/orders/fulfill" config-ref="HTTP_Listener_config"/>
  <s3:upload config-ref="S3_Config" doc:name="Archive order document"
    bucketName="order-archive" key="#[vars.orderId ++ '.json']">
    <s3:content>#[payload]</s3:content>
  </s3:upload>
  <lambda:invoke config-ref="Lambda_Config" doc:name="Reserve inventory"
    functionName="reserve-inventory" invocationType="RequestResponse"
    target="reservation"/>
  <dynamodb:put-item config-ref="DynamoDB_Config" doc:name="Record fulfillment state"
    tableName="order-fulfillment">
    <dynamodb:item>#{
      orderId: { S: vars.orderId },
      status: { S: 'INVENTORY_RESERVED' },
      reservationId: { S: vars.reservation.reservationId },
      fulfilledAt: { S: now() as String }
    }</dynamodb:item>
  </dynamodb:put-item>
  <try>
    <sqs:publish config-ref="SQS_Config" doc:name="Notify warehouse"
      queueUrl="${sqs.warehouseNotifyQueueUrl}"
      content="#[{ orderId: vars.orderId, reservationId:
        vars.reservation.reservationId }]"/>
    <error-handler>
      <on-error-continue type="AWS_SQS:CONNECTIVITY">
        <logger level="WARN" message="Warehouse notification failed – fulfillment
          already recorded, will not block response"/>
      </on-error-continue>
    </error-handler>
  </try>
</flow>

```

EXPLANATION OF THE CODE

`dynamodb:put-item` runs only after `lambda:invoke` has returned successfully

Sequencing the state write strictly after the Lambda call means the fulfillment record can never claim inventory was reserved when the reservation call itself actually failed — an unhandled Lambda error here propagates and the flow stops before DynamoDB is ever touched.

`on-error-continue` around only the `SQS publish`

Isolating error handling to just the notification step, rather than the whole flow, reflects that a failed warehouse notification shouldn't undo

or block an order that's already been fulfilled — the fulfillment itself is already durably recorded in DynamoDB by this point.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener**, then the S3 **Upload** operation to archive the order document.
- 2 Add **AWS Lambda > Invoke** for reserve-inventory, target variable reservation.
- 3 Add the DynamoDB **Put Item** operation writing the fulfillment state record.
- 4 Wrap the SQS **Publish** operation in a **Try** scope with a non-blocking error handler, then save and run.

TESTING IN POSTMAN

POST

https://localhost:8081/orders/fulfill

SAMPLE REQUEST BODY

JSON

```
{ "orderId": "ORD-2201", "sku": "SKU-4471", "quantity": 2 }
```

SAMPLE RESPONSE

JSON

```
{ "orderId": "ORD-2201", "status": "INVENTORY_RESERVED" }
```

BRIEFING ALL THE STEPS IN WORDS

The S3 Connector's Upload operation archives the order document, the Lambda Connector's Invoke operation reserves inventory synchronously, the DynamoDB Connector's Put Item operation records the fulfillment state only once that reservation has succeeded, and the SQS Connector's Publish operation notifies the warehouse as a final, non-blocking step — four AWS services orchestrated into one order-fulfillment process where the sequencing itself is what keeps the recorded state trustworthy.

#82 Sending Transactional Emails via Amazon SES from a Mule Flow

AWS

PROBLEM STATEMENT

Once an order ships, the customer needs a shipping-confirmation email with the tracking number, sent immediately and reliably, without standing up a separate email service.

DETAILED EXPLANATION

SES handles the actual mail delivery, bounce handling, and sender reputation, so the Mule flow's job is just to build the message and hand it off — either as a simple templated email or, for richer formatting, an HTML body alongside a plain-text fallback. Using an SES template, configured once and referenced by name, keeps the actual email markup out of the Mule flow entirely, so a marketing or CX team can update the email's wording without a Mule deployment.

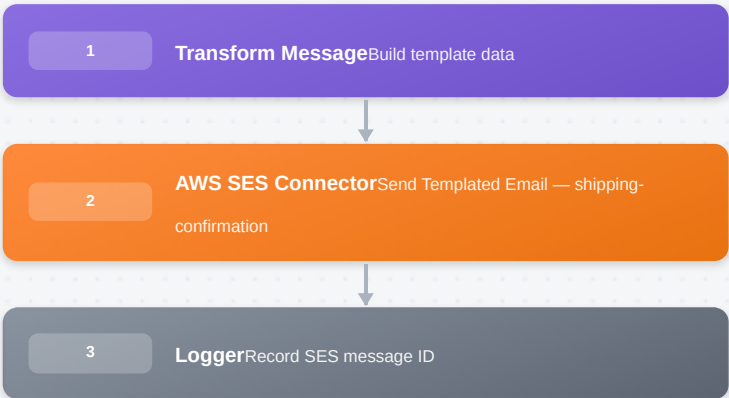
CONNECTORS & COMPONENTS USED

AWS AWS SES Connector – Send Templated Email

STEPS TO CREATE THE MULE FLOW

- 1 After a shipment is confirmed, build the template data with the customer's name and tracking number.
- 2 Call SES Send Templated Email using the pre-registered shipping-confirmation template.
- 3 Log the SES message ID for delivery tracking.

SEND-SHIPPING-CONFIRMATION-FLOW



MULE CONFIGURATION XML

```

<flow name="send-shipping-confirmation-flow">
  <ee:transform doc:name="Build template data">
    <ee:variables>
      <ee:set-variable variableName="templateData">#[write({
customerName: payload.customerName,
trackingNumber: payload.trackingNumber,
carrier: payload.carrier
}, "application/json")]</ee:set-variable>
    </ee:variables>
  </ee:transform>
  <ses:send-templated-email config-ref="SES_Config" doc:name="Send shipping
confirmation"
    source="orders@example.com" templateName="shipping-confirmation"
    templateData="#[vars.templateData]" target="sesResult">
    <ses:destination>
      <ses:to-addresses>#[[payload.customerEmail]]</ses:to-addresses>
    </ses:destination>
  </ses:send-templated-email>
  <logger level="INFO" message="#[ 'Shipping confirmation sent, messageId=' ++
vars.sesResult.messageId]"/>
</flow>

```

EXPLANATION OF THE CODE

`templateName="shipping-confirmation"` with `templateData` as a JSON string

SES resolves the actual subject and body from the named template registered in SES itself — this call only supplies the variable data the template's placeholders reference, so the wording, branding, and layout of the email live entirely outside the Mule application.

`vars.sesResult.messageId`

SES returns a unique message ID for every accepted send, which is worth logging or storing — it's the identifier needed to look up that specific email's delivery, bounce, or complaint status later if the customer says they never received it.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Add a **Transform Message** building the `templateData` JSON string from the shipment payload.
- 2 Drag **AWS SES > Send Templated Email** and set **Source**, **Template Name**, and **Template Data**.
- 3 Set the **Destination**'s To Addresses to the customer's email.
- 4 Save and run, then confirm the SES message ID appears in the log.

TESTING IN POSTMAN

POST

https://localhost:8081/orders/notify-shipped

SAMPLE REQUEST BODY

JSON

```
{ "customerEmail": "j.alvarez@example.com", "customerName": "J. Alvarez", "trackingNumber": "1Z999AA10123456784", "carrier": "UPS" }
```

SAMPLE RESPONSE

JSON

```
{ "sent": true, "messageId": "0102018c9f7e-example-message-id" }
```

BRIEFING ALL THE STEPS IN WORDS

A transform assembles the customer name, tracking number, and carrier into the template's expected data shape, and the SES Connector's Send Templated Email operation hands that data off to a pre-registered SES template for the actual send — keeping the email's wording and layout owned outside the Mule application while the flow logs SES's returned message ID for later delivery tracking.

#83 Loading Transformed Data into Amazon Redshift from Mule

AWS

PROBLEM STATEMENT

A nightly job needs to load a day's worth of cleaned, transformed order records into a Redshift analytics table, in a way that scales past what row-by-row inserts can handle.

DETAILED EXPLANATION

Redshift is built for bulk analytical loads, not transactional row-by-row writes, and the standard, high-throughput way to get data in is the COPY command reading from S3 rather than individual INSERT statements. This flow writes the day's transformed records as a single file to a staging S3 location, then issues a COPY command through Redshift's Data API to load that file into the target table in one bulk operation.

CONNECTORS & COMPONENTS USED

AWS

AWS S3 Connector – Upload

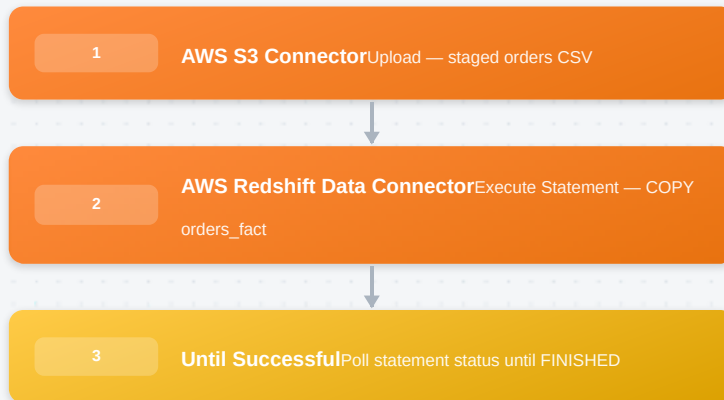
AWS

AWS Redshift Data Connector – Execute Statement

STEPS TO CREATE THE MULE FLOW

- 1 Transform the day's order records to CSV and upload the file to a Redshift staging bucket.
- 2 Execute a COPY statement through the Redshift Data API pointing at that staged file.
- 3 Poll the statement's execution status until it completes and log the row count loaded.

LOAD-ORDERS-TO-REDSHIFT-FLOW



MULE CONFIGURATION XML

```
<flow name="load-orders-to-redshift-flow">
  <s3:upload config-ref="S3_Config" doc:name="Stage orders CSV"
    bucketName="redshift-staging" key="#['orders/' ++ (now() as String {format:
'yyyy-MM-dd'}) ++ '.csv'}]">
    <s3:content>#[write(payload, "application/csv")]</s3:content>
  </s3:upload>
  <redshift-data:execute-statement config-ref="RedshiftData_Config" doc:name="Copy
staged file into orders_fact"
    workgroupName="analytics-workgroup" database="analytics"
    sql="#['COPY orders_fact FROM \'s3://redshift-staging/orders/' ++ (now() as
String {format: "yyyy-MM-dd'}) ++ '.csv\' IAM_ROLE
\'arn:aws:iam::998877665544:role/redshift-copy-role\' CSV IGNOREHEADER 1']"
    target="copyStatement"/>
  <until-successful maxRetries="10" millisBetweenRetries="5000" doc:name="Poll for
completion">
    <redshift-data:describe-statement config-ref="RedshiftData_Config" doc:name="Check
COPY status"
      statementId="#[vars.copyStatement.id]" target="copyStatus"/>
    <validation:is-true expression="#[vars.copyStatus.status == 'FINISHED']"
      message="Redshift COPY still running"/>
  </until-successful>
  <logger level="INFO" message="#['Redshift COPY finished, rows loaded: ' ++
(vars.copyStatus.resultRows as String)]"/>
</flow>
```

EXPLANATION OF THE CODE

```
COPY orders_fact FROM 's3://...' IAM_ROLE '...'
```

Redshift's COPY command loads directly from S3 in parallel across the cluster's nodes, which is dramatically faster than issuing one

INSERT per row through a JDBC connection — the IAM_ROLE clause is how Redshift authenticates to S3 to read the staged file, rather than embedding credentials in the SQL itself.

```
until-successful polling describe-statement for status == 'FINISHED'
```

Execute Statement against the Redshift Data API is asynchronous — it kicks the COPY off and returns a statement ID immediately, so the flow has to separately poll Describe Statement until the load actually completes before it can safely report a row count or move on.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag the S3 Upload operation to stage the day's transformed CSV into the staging bucket.
- 2 Add AWS Redshift Data > Execute Statement with the COPY SQL referencing the staged file's S3 path.
- 3 Wrap a Redshift Data > Describe Statement operation and a Validation > Is True check inside an Until Successful scope.
- 4 Save and run, then confirm the row count logs correctly once the COPY finishes.

TESTING IN POSTMAN

Scheduler-triggered nightly. To test on demand, expose an admin endpoint invoking the same subflow.

POST https://localhost:8081/admin/load-orders-to-redshift

SAMPLE RESPONSE

JSON

```
{ "status": "FINISHED", "rowsLoaded": 18420 }
```

BRIEFING ALL THE STEPS IN WORDS

The S3 Connector's Upload operation stages the day's transformed order data as a single CSV file, the Redshift Data Connector's Execute Statement operation kicks off a COPY command loading that file directly into the analytics table in bulk, and an Until Successful scope polls Describe Statement until the load reports finished — giving the nightly job a scalable path into Redshift that doesn't depend on row-by-row inserts.

#84 Triggering an AWS Glue ETL Job from a Mule Flow and Monitoring Its Status

AWS

PROBLEM STATEMENT

Once the day's raw order files have landed in S3, a Glue ETL job needs to run to clean, deduplicate, and partition them into the data lake's curated layer — kicked off by the same Mule flow that finished landing the files, rather than on its own independent schedule

that might race ahead of the data.

DETAILED EXPLANATION

Starting a Glue job through the Glue connector rather than a fixed Glue-side schedule means the ETL only runs once the upstream landing step has actually finished — eliminating the class of bug where a scheduled Glue job kicks off while the last of the day's files are still being written. Glue jobs run asynchronously and can take anywhere from minutes to hours, so this flow starts the run and polls its status rather than waiting synchronously.

CONNECTORS & COMPONENTS USED

AWS

AWS Glue Connector – Start Job Run

AWS

AWS Glue Connector – Get Job Run

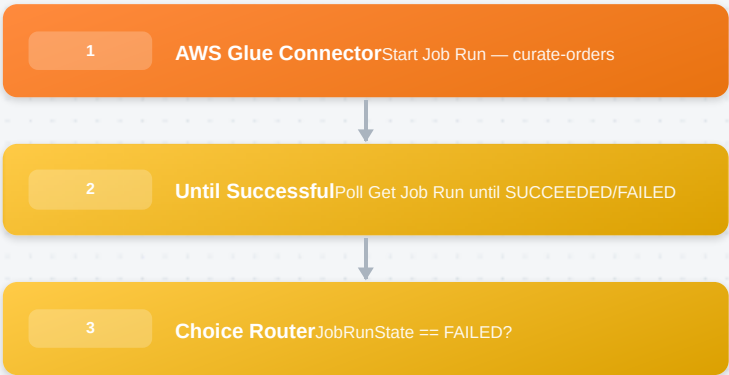
ERR

Until Successful

STEPS TO CREATE THE MULE FLOW

- 1
- After the day's files finish landing, start the curate-orders Glue job run.
- 2
- Poll the job run's status until it reports SUCCEEDED or FAILED.
- 3
- On failure, raise an error that the surrounding pipeline's alerting can catch.

RUN-CURATE-ORDERS-ETL-FLOW



MULE CONFIGURATION XML

```

<flow name="run-curate-orders-etl-flow">
  <glue:start-job-run config-ref="Glue_Config" doc:name="Start curate-orders job"
    jobName="curate-orders" target="jobRun"/>
  <until-successful maxRetries="60" millisBetweenRetries="30000" doc:name="Poll job
status">
    <glue:get-job-run config-ref="Glue_Config" doc:name="Check job run"
      jobName="curate-orders" runId="#[vars.jobRun.jobRunId]" target="jobStatus"/>
    <validation:is-true expression="#([('SUCCEEDED', 'FAILED', 'STOPPED']) contains
vars.jobStatus.jobRunState]"
      message="Glue job still running"/>
  </until-successful>
  <choice doc:name="Job succeeded?">
    <when expression="#[vars.jobStatus.jobRunState != 'SUCCEEDED']">
      <raise-error type="APP:ETL_FAILED" description="#['Glue job curate-orders ended
in state ' ++ vars.jobStatus.jobRunState]"/>
    </when>
  </choice>
</flow>

```

EXPLANATION OF THE CODE

`maxRetries="60" millisBetweenRetries="30000"`

A 30-second poll interval up to 60 times gives the job up to half an hour to finish, which is a reasonable ceiling for a daily curation job — a much longer-running Glue job would instead be a candidate for the CloudWatch Events / EventBridge job-completion trigger pattern rather than polling this long from within a single flow.

`(['SUCCEEDED', 'FAILED', 'STOPPED']) contains vars.jobStatus.jobRunState`

The Until Successful scope needs a condition that's satisfied by any terminal state, not just success — otherwise a job that actually failed would keep getting re-pollled as if it were still in progress, right up until retries were exhausted, rather than being detected and reported immediately.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag `AWS Glue > Start Job Run` and set `Job Name` to `curate-orders`.
- 2 Wrap `Glue > Get Job Run` and a `Validation > Is True` check inside an `Until Successful` scope, 60 retries at 30 seconds apart.
- 3 Add a `Choice` router checking the final `jobRunState`, with a `Raise Error` in the non-success branch.
- 4 Save and run, and watch the job's progress in the Glue console alongside the Mule log.

TESTING IN POSTMAN

Chained internally after the landing flow. To test directly, expose an admin endpoint invoking this subflow.

POST `https://localhost:8081/admin/run-curate-orders-etl`

SAMPLE RESPONSE

JSON

```
{ "jobName": "curate-orders", "jobRunState": "SUCCEEDED" }
```

BRIEFING ALL THE STEPS IN WORDS

The Glue Connector's Start Job Run operation kicks off the curation job right after the day's files finish landing, an Until Successful scope polls Get Job Run every 30 seconds until the run reaches a terminal state, and a Choice router raises a clear error if that terminal state isn't SUCCEEDED — making the ETL step a dependable, monitored link in the pipeline rather than a fire-and-forget schedule that might run against incomplete data.

#85 Invalidating a CloudFront Cache After Publishing New Content to S3

AWS

PROBLEM STATEMENT

A product-catalog JSON file served through CloudFront in front of an S3 origin bucket was just updated by an upstream PIM sync flow, but CloudFront's edge caches will keep serving the old version for up to an hour unless explicitly invalidated.

DETAILED EXPLANATION

CloudFront caches objects at edge locations to reduce load on the S3 origin and improve latency for end users, but that means a fresh S3 write alone doesn't guarantee fresh content is served — an explicit invalidation request tells CloudFront to purge that specific object path from every edge location so the next request re-fetches it from the origin. Invalidating narrowly by exact path, rather than a broad wildcard, keeps the operation cheap and avoids purging unrelated cached content unnecessarily.

CONNECTORS & COMPONENTS USED

AWS

AWS S3 Connector – Upload

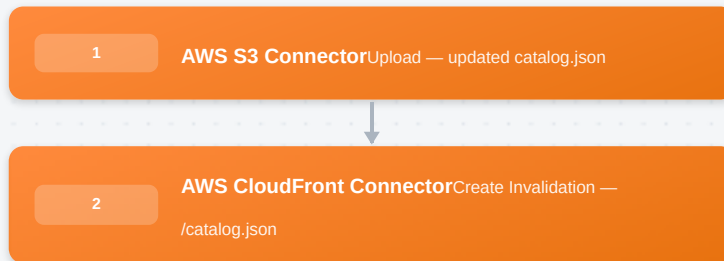
AWS

AWS CloudFront Connector – Create Invalidation

STEPS TO CREATE THE MULE FLOW

- 1 Upload the updated catalog JSON to the origin S3 bucket.
- 2 Create a CloudFront invalidation for that object's exact path.
- 3 Log the invalidation ID for tracking, without waiting for it to complete.

PUBLISH-CATALOG-AND-INVALIDATE-FLOW



MULE CONFIGURATION XML

```
<flow name="publish-catalog-and-invalidate-flow">
  <s3:upload config-ref="S3_Config" doc:name="Publish updated catalog"
    bucketName="catalog-origin-bucket" key="catalog.json">
    <s3:content>#[write(payload, "application/json")]</s3:content>
  </s3:upload>
  <cloudfront:create-invalidation config-ref="CloudFront_Config" doc:name="Purge cached
catalog">
    distributionId="E1A2B3C4D5E6F7" callerReference="#['catalog-update-' ++ (now() as
Number {unit: "seconds"} as String)]">
    <cloudfront:paths>#[[ "/catalog.json" ]]</cloudfront:paths>
  </cloudfront:create-invalidation>
</flow>
```

EXPLANATION OF THE CODE

`callerReference` built from the current timestamp

CloudFront requires each invalidation request to carry a unique caller reference to prevent accidental duplicate submissions from being treated as one — using a timestamp-derived value here is a simple way to guarantee uniqueness across repeated runs of this flow.

`cloudfront:paths > [ "/catalog.json" ]`

Scoping the invalidation to this one exact path, instead of a wildcard like `/*`, purges only the object that actually changed — a broad wildcard invalidation is both more expensive (CloudFront bills per path beyond a monthly free tier) and briefly increases origin load across the entire site, not just this one file.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag the S3 `Upload` operation and set `Key` to `catalog.json`.
- 2 Drag `AWS CloudFront > Create Invalidation` and set `Distribution Id` to the catalog distribution's ID.
- 3 Set `Caller Reference` to a unique, timestamp-based expression, and `Paths` to the exact object path.

- 4 Save and run, then confirm the edge-served response reflects the new content within a minute or two.

TESTING IN POSTMAN

POST `https://localhost:8081/catalog/publish`

SAMPLE RESPONSE

JSON

```
{ "published": true, "invalidationId": "I2J3K4L5M6N7O8" }
```

BRIEFING ALL THE STEPS IN WORDS

The S3 Connector's Upload operation writes the updated catalog file to the CloudFront distribution's origin bucket, and the CloudFront Connector's Create Invalidation operation immediately purges that exact path from every edge location with a uniquely referenced request — ensuring the next request for the catalog anywhere in the world gets the freshly published version instead of whatever was cached before the update.

#86 Triggering a Mule Flow from an ECS/Fargate Task-Completion Event

AWS

PROBLEM STATEMENT

A video-transcoding job runs as a short-lived Fargate task; when it finishes, downstream systems need to know so they can publish the transcoded asset. Rather than polling ECS for task status, the Mule flow should react the moment the task actually stops.

DETAILED EXPLANATION

ECS automatically emits a task state-change event to EventBridge whenever a task's status changes, including the transition to STOPPED — so instead of a Mule flow polling ECS's DescribeTasks API on a timer, an EventBridge rule matching that specific event pattern routes it to an SQS queue the flow listens on. This reacts within seconds of the task actually finishing and, unlike polling, costs nothing while the task is still running.

CONNECTORS & COMPONENTS USED

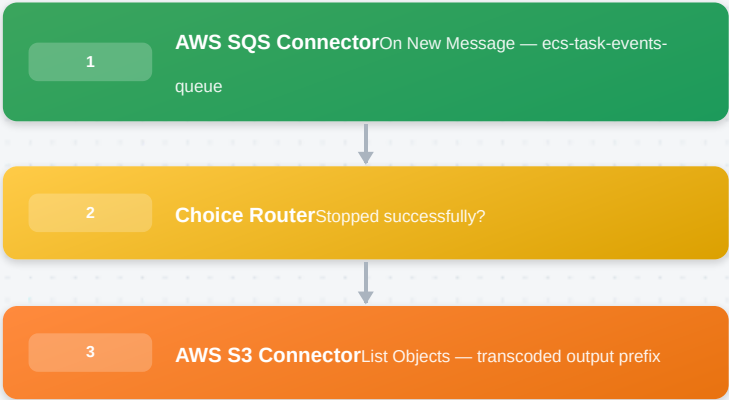
AWS AWS SQS Connector – On New Message

AWS AWS S3 Connector – List Objects

STEPS TO CREATE THE MULE FLOW

- 1 Listen on the SQS queue that receives ECS task state-change events via an EventBridge rule.
- 2 Check that the event's lastStatus is STOPPED and stopCode is not a failure code.
- 3 Locate the transcoded output in S3 and publish it downstream.

HANDLE-TRANSCODE-TASK-COMPLETE-FLOW



MULE CONFIGURATION XML


```

<flow name="handle-transcode-task-complete-flow">
  <sqs:listener config-ref="SQS_Config" queueUrl="${sqs.ecsTaskEventsQueueUrl}"
ackMode="MANUAL"/>
  <ee:transform doc:name="Parse ECS task event">
    <ee:variables>
      <ee:set-variable variableName="taskEvent">#[read(payload, "application/json")]
    </ee:set-variable>
    </ee:variables>
  </ee:transform>
  <choice doc:name="Stopped successfully?">
    <when expression="#[vars.taskEvent.detail.lastStatus == 'STOPPED' and
vars.taskEvent.detail.stopCode == 'EssentialContainerExited'
      and (vars.taskEvent.detail.containers[0]).exitCode == 0]">
      <s3:list-objects config-ref="S3_Config" doc:name="Find transcoded output"
        bucketName="transcoded-assets"
        prefix="#['output/' ++
vars.taskEvent.detail.overrides.containerOverrides[0].environment[0].value]"
        target="outputListing"/>
      <logger level="INFO" message="Transcode task complete, output located, ready to
publish"/>
    </when>
    <otherwise>
      <logger level="WARN" message="Transcode task did not stop successfully –
investigate"/>
    </otherwise>
  </choice>
  <sqs:delete-message config-ref="SQS_Config" queueUrl="${sqs.ecsTaskEventsQueueUrl}"
    receiptHandle="#[attributes.receiptHandle]"/>
</flow>

```

EXPLANATION OF THE CODE

`stopCode == 'EssentialContainerExited' and exitCode == 0`

A task reaching STOPPED status doesn't by itself mean it succeeded — ECS uses the same lastStatus for a task that crashed, was manually stopped, or ran out of memory, so checking the stop code and the essential container's actual exit code is what distinguishes a genuine successful completion from a failure that merely also stopped.

`prefix built from a container override environment variable`

The original task launch passed the output location as a container environment variable, and that same value is echoed back in the task's detail payload — reading it back out here is what lets this flow find the correct output location without needing a

separate lookup or a naming convention based purely on the task ID.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag the SQS **On New Message** source pointed at the queue subscribed to the ECS task-event EventBridge rule.
- 2 Add a **Transform Message** parsing the event payload into a taskEvent variable.
- 3 Add a **Choice** router checking lastStatus, stopCode, and the container's exit code together.
- 4 In the success branch, add the S3 **List Objects** operation, then a **Delete Message** operation after the Choice, and save and run.

TESTING IN POSTMAN

SQS-triggered via EventBridge. Publish a sample ECS task state-change payload to the queue to test end to end.

POST `https://localhost:8081/admin/simulate-ecs-task-event`

SAMPLE RESPONSE

JSON

```
{ "eventHandled": true, "outputFound": true }
```

BRIEFING ALL THE STEPS IN WORDS

The SQS Connector's On New Message source delivers ECS task state-change events routed through an EventBridge rule the moment a transcoding task stops, a Choice router checks the stop code and container exit code to confirm the task actually succeeded rather than merely stopped, and the S3 Connector's List Objects operation locates the resulting transcoded output — reacting to task completion in seconds instead of polling ECS on a timer.

#87 Submitting and Monitoring an AWS Batch Job from Mule

AWS

PROBLEM STATEMENT

A monthly reconciliation report requires running a compute-heavy job over millions of transaction records — too large and long-running for Lambda's execution time limit — and the flow needs to submit it as an AWS Batch job and track it through to completion.

DETAILED EXPLANATION

AWS Batch is designed for exactly this kind of workload: it provisions the compute resources on demand, queues the job, and runs it to completion without the 15-minute ceiling Lambda imposes. Submitting a job from Mule looks similar to the Step Functions pattern from Volume 3 — start, then poll — but Batch jobs pass through several intermediate states (SUBMITTED, RUNNABLE, STARTING, RUNNING) before reaching a terminal one, and a production flow generally cares only about distinguishing SUCCEEDED from FAILED, treating everything else as still in progress.

CONNECTORS & COMPONENTS USED

AWS

AWS Batch Connector – Submit Job

AWS

AWS Batch Connector – Describe Jobs

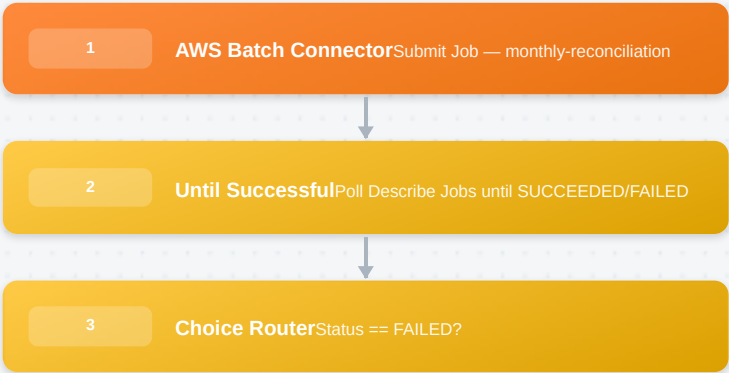
ERR

Until Successful

STEPS TO CREATE THE MULE FLOW

- 1 Submit the reconciliation job to the designated Batch job queue and definition.
- 2 Poll Describe Jobs until the job reaches SUCCEEDED or FAILED.
- 3 On FAILED, capture the container's exit reason and raise an error.

RUN-MONTHLY-RECONCILIATION-FLOW



MULE CONFIGURATION XML

```

<flow name="run-monthly-reconciliation-flow">
  <batch-aws:submit-job config-ref="Batch_Config" doc:name="Submit reconciliation job"
    jobName="#['monthly-reconciliation-' ++ (now() as String {format: 'yyyy-MM'})]"
    jobQueue="reconciliation-job-queue" jobDefinition="reconciliation-job-def"
    target="batchJob"/>
  <until-successful maxRetries="120" millisBetweenRetries="15000" doc:name="Poll job
status">
    <batch-aws:describe-jobs config-ref="Batch_Config" doc:name="Check job status"
      jobs="#[vars.batchJob.jobId]" target="jobStatus"/>
    <validation:is-true expression="#(['SUCCEEDED', 'FAILED']) contains
(vars.jobStatus.jobs[0]).status]"
      message="Batch job still in progress"/>
  </until-successful>
  <choice doc:name="Job failed?">
    <when expression="#[(vars.jobStatus.jobs[0]).status == 'FAILED']">
      <raise-error type="APP:BATCH_JOB_FAILED"
        description="#['Reconciliation job failed: ' ++
((vars.jobStatus.jobs[0]).container.reason default 'no reason given')]">
    </when>
  </choice>
</flow>

```

EXPLANATION OF THE CODE

`maxRetries="120" millisBetweenRetries="15000"`

Fifteen-second polling for up to half an hour reflects that a heavy monthly reconciliation job can genuinely take a while once it starts running — the poll interval is intentionally coarser than the ECS example's near-real-time need, since nothing downstream depends on learning about completion within seconds.

`(vars.jobStatus.jobs[0]).container.reason default 'no reason given'`

When a Batch job's container exits with a non-zero code, AWS Batch often populates a human-readable reason describing what happened, which is worth surfacing directly in the raised error — far more useful for triage than just knowing the job failed.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag `AWS Batch > Submit Job` and set `Job Queue` and `Job Definition`.
- 2 Wrap `Batch > Describe Jobs` and a `Validation > Is True` check in an `Until Successful` scope, 120 retries at 15 seconds apart.
- 3 Add a `Choice` router checking the final status, with a `Raise Error` surfacing the container's exit reason on failure.

- 4 Save and run, then watch the job's progress through its states in the AWS Batch console.

TESTING IN POSTMAN

Scheduler-triggered monthly. To test on demand, expose an admin endpoint invoking this subflow.

POST `https://localhost:8081/admin/run-reconciliation-now`

SAMPLE RESPONSE

JSON

```
{ "jobName": "monthly-reconciliation-2026-08", "status": "SUCCEEDED" }
```

BRIEFING ALL THE STEPS IN WORDS

The AWS Batch Connector's Submit Job operation queues the reconciliation job with the compute resources it needs, an Until Successful scope polls Describe Jobs at a deliberately coarse interval until the job reaches SUCCEEDED or FAILED, and a Choice router raises a clear, reason-carrying error on failure — giving a job too large and long-running for Lambda a dependable path to run and be monitored entirely from within the flow that needs its result.

#88 Building a Hybrid Integration Syncing On-Prem Data to AWS RDS

AWS

PROBLEM STATEMENT

A legacy on-prem inventory database must stay in sync with an RDS PostgreSQL instance that a new cloud application reads from, with changes flowing one-way from on-prem to RDS on a five-minute cadence, without replacing the legacy system.

DETAILED EXPLANATION

Rather than a full table dump every cycle, this flow does an incremental sync based on a last-modified timestamp column already present in the on-prem schema — pulling only rows changed since the last successful sync, and upserting them into RDS so a record updated twice between syncs still lands correctly. The Mule worker connects to the on-prem database through Anypoint VPN or a database gateway and to RDS directly, making this flow the bridge itself rather than either side needing direct network visibility to the other.

CONNECTORS & COMPONENTS USED

CORE Database Connector (on-prem)

CORE Database Connector (RDS)

CORE Scheduler

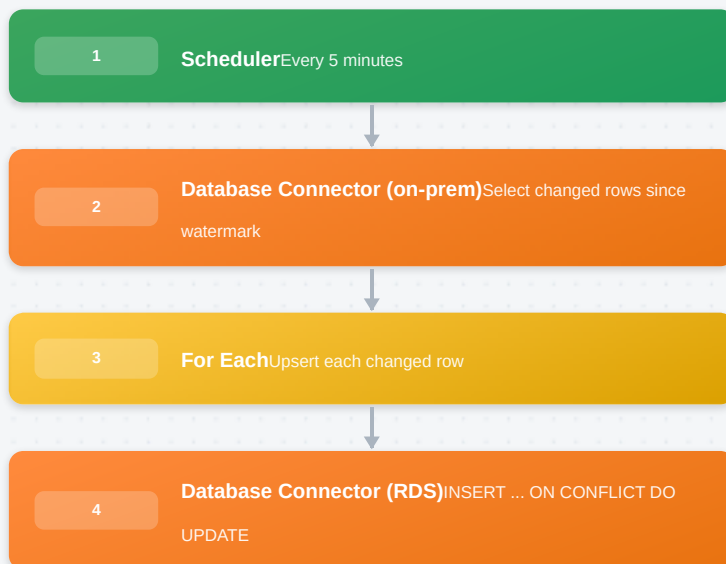
CORE Object Store – Last Sync Watermark

STEPS TO CREATE THE MULE FLOW

- 1 Read the last successful sync watermark from Object Store.
- 2 Query the on-prem database for rows modified after that watermark.

- 3 Upsert each row into RDS and advance the watermark only after the batch succeeds.

SYNC-INVENTORY-TO-RDS-FLOW



MULE CONFIGURATION XML

```

<flow name="sync-inventory-to-rds-flow">
  <scheduler doc:name="Every 5 minutes">
    <scheduling-strategy>
      <fixed-frequency frequency="5" timeUnit="MINUTES"/>
    </scheduling-strategy>
  </scheduler>
  <os:retrieve objectStore="syncWatermarkStore" key="lastSyncedAt" target="watermark"/>
  <db:select config-ref="OnPrem_DB_Config" doc:name="Select changed inventory rows"
target="changedRows">
    <db:sql>SELECT sku, quantity, last_modified FROM inventory WHERE last_modified >
:watermark</db:sql>
    <db:input-parameters>#{[ watermark: vars.watermark ]}</db:input-parameters>
  </db:select>
  <foreach collection="#[vars.changedRows]" doc:name="Upsert each row into RDS">
    <db:bulk-insert config-ref="RDS_Config" doc:name="Upsert inventory row">
      <db:sql>INSERT INTO inventory (sku, quantity, last_modified) VALUES (:sku,
:quantity, :lastModified)
      ON CONFLICT (sku) DO UPDATE SET quantity = EXCLUDED.quantity, last_modified =
EXCLUDED.last_modified</db:sql>
      <db:input-parameters>#{[ sku: payload.sku, quantity: payload.quantity,
lastModified: payload.last_modified ]}</db:input-parameters>
    </db:bulk-insert>
  </foreach>
  <os:store objectStore="syncWatermarkStore" key="lastSyncedAt">
    <os:value>#[now()]</os:value>
  </os:store>
</flow>

```

EXPLANATION OF THE CODE

INSERT ... ON CONFLICT (sku) DO UPDATE

PostgreSQL's upsert syntax means this sync is naturally idempotent — re-running the same batch, or a row appearing again in a later sync window because it changed twice, simply overwrites the existing row rather than producing a duplicate-key error or a duplicate record.

os:store for lastSyncedAt runs only after the entire foreach completes

Advancing the watermark only once every row in the batch has been successfully upserted means a failure partway through the loop leaves the watermark exactly where it was, so the next scheduled run naturally re-attempts the same batch rather than silently skipping the rows that failed.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag a **Scheduler** and configure a 5-minute fixed frequency.
- 2 Add an **Object Store > Retrieve** for the watermark, then a **Database > Select** operation against the on-prem connection using it.
- 3 Add a **For Each** over the changed rows with a **Database > Bulk Insert** operation using an upsert statement against the RDS connection.
- 4 Add an **Object Store > Store** operation after the loop to advance the watermark, then save and run.

TESTING IN POSTMAN

Scheduler-triggered. To test on demand, expose an admin endpoint invoking this subflow.

POST `https://localhost:8081/admin/sync-inventory-now`

SAMPLE RESPONSE

```
{ "rowsSynced": 37, "newWatermark": "2026-08-28T09:45:00Z" }
```

BRIEFING ALL THE STEPS IN WORDS

The Database Connector queries the on-prem inventory table for rows changed since the last recorded watermark, a For Each loop upserts each one into RDS using PostgreSQL's ON CONFLICT syntax so the sync stays idempotent, and the watermark advances only once the whole batch has succeeded — keeping a cloud application's read replica current with a legacy on-prem system every five minutes without either side needing to know the other exists beyond this one Mule flow.

#89 Implementing an Idempotent AWS Integration to Safely Handle Duplicate Events

AWS

PROBLEM STATEMENT

SQS's at-least-once delivery guarantee means the same order-created message can occasionally be delivered twice. The flow that charges the customer's card and writes the order record must not double-charge or double-write when that happens.

DETAILED EXPLANATION

Idempotency here means designing the flow so that processing the same message twice produces the same end state as processing it once — not by hoping duplicates never arrive, since SQS explicitly doesn't guarantee that, but by checking, before doing anything irreversible, whether this exact message's idempotency key has already been processed. A DynamoDB table with a conditional write acts as the guard: the first delivery successfully claims the key and proceeds, and a second delivery's conditional write fails, telling the flow to skip the charge and the write entirely.

CONNECTORS & COMPONENTS USED

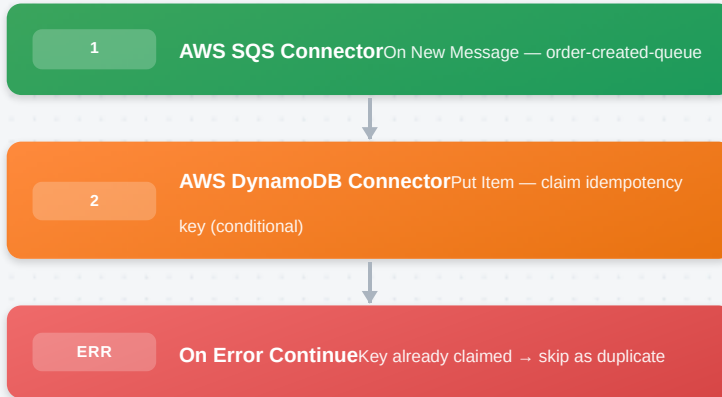
AWS **AWS SQS Connector – On New Message**

AWS **AWS DynamoDB Connector – Put Item (conditional)**

STEPS TO CREATE THE MULE FLOW

- 1 On message receipt, attempt a conditional DynamoDB write claiming the message's idempotency key.
- 2 If the claim succeeds, charge the card and process the order.
- 3 If the claim fails because the key already exists, log it as a duplicate and skip processing entirely.

PROCESS-ORDER-IDEMPOTENTLY-FLOW



MULE CONFIGURATION XML

```

<flow name="process-order-idempotently-flow">
  <sqs:listener config-ref="SQS_Config" queueUrl="${sqs.orderCreatedQueueUrl}"
ackMode="MANUAL"/>
  <ee:transform doc:name="Parse order event">
    <ee:variables>
      <ee:set-variable variableName="order">#[read(payload, "application/json")]
    </ee:set-variable>
    </ee:variables>
  </ee:transform>
  <try>
    <dynamodb:put-item config-ref="DynamoDB_Config" doc:name="Claim idempotency key"
tableName="processed-order-events"
      conditionExpression="attribute_not_exists(idempotencyKey)">
      <dynamodb:item>#[{ idempotencyKey: { S: vars.order.eventId }, processedAt: { S:
now() as String } }]/>
    </dynamodb:put-item>
    <flow-ref name="charge-card-and-record-order-subflow"/>
    <error-handler>
      <on-error-continue type="AWS_DYNAMODB_CONDITIONAL_CHECK_FAILED">
        <logger level="INFO" message="#[ 'Duplicate delivery of eventId ' ++
vars.order.eventId ++ ' - skipping' ]"/>
      </on-error-continue>
    </error-handler>
  </try>
  <sqs:delete-message config-ref="SQS_Config" queueUrl="${sqs.orderCreatedQueueUrl}"
receiptHandle="#[attributes.receiptHandle]"/>
</flow>

```

EXPLANATION OF THE CODE

`conditionExpression="attribute_not_exists(idempotencyKey)"`

This condition makes the write itself the mechanism that decides whether this event has already been seen — DynamoDB rejects the write if a row with that key already exists, so there's no separate read-then-write race window where two concurrent deliveries could both check, see nothing, and both proceed.

`flow-ref` only runs if the claiming `put-item` succeeded

Placing the actual charge-and-record logic immediately after the conditional write, inside the same Try scope, guarantees the irreversible work only happens once the key has been durably claimed — a duplicate delivery never gets far enough to reach the card charge at all.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag the SQS **On New Message** source for order-created-queue and add a **Transform Message** parsing the event.
- 2 Inside a **Try** scope, add the DynamoDB **Put Item** operation with **Condition Expression** set to `attribute_not_exists(idempotencyKey)`.
- 3 Add a **Flow Reference** to the charge-and-record subflow right after it.
- 4 Add an **On Error Continue** handler for `AWS_DYNAMODB:CONDITIONAL_CHECK_FAILED` logging the duplicate, then a **Delete Message** operation, and save and run.

TESTING IN POSTMAN

SQS-triggered. Publish the same event twice to the queue to confirm the second delivery is skipped.

POST `https://localhost:8081/admin/simulate-duplicate-order-event`

SAMPLE RESPONSE

```
{ "firstDelivery": "processed", "secondDelivery": "duplicate_skipped" }
```

BRIEFING ALL THE STEPS IN WORDS

The DynamoDB Connector's Put Item operation attempts a conditional write that only succeeds if this exact event's idempotency key hasn't been claimed before, the card charge and order record only run once that claim succeeds, and an On Error Continue handler quietly logs and skips any delivery that loses that race — making the flow safe against SQS's at-least-once delivery guarantee without ever needing to trust that duplicates simply won't happen.

#90 Designing a Multi-Region Failover Strategy for a MuleSoft-AWS Integration

AWS

PROBLEM STATEMENT

The order-processing integration currently depends entirely on us-east-1; a regional AWS outage there would take down order processing company-wide. The integration needs a documented, largely automated path to fail over to us-west-2 without a lengthy manual rebuild.

DETAILED EXPLANATION

Multi-region failover for this kind of integration rests on a few concrete pieces working together: DynamoDB Global Tables replicating the order state across both regions automatically, S3 Cross-Region Replication keeping archived documents available in both, a Mule worker deployed and idling in the secondary region, and Route 53 health checks that detect the primary region's failure and shift DNS traffic to the secondary. The Mule-side responsibility is narrower than it might sound: the flow logic itself doesn't change between regions at all — it's the same application deployed twice, pointed at region-appropriate AWS endpoints through environment-specific properties.

CONNECTORS & COMPONENTS USED

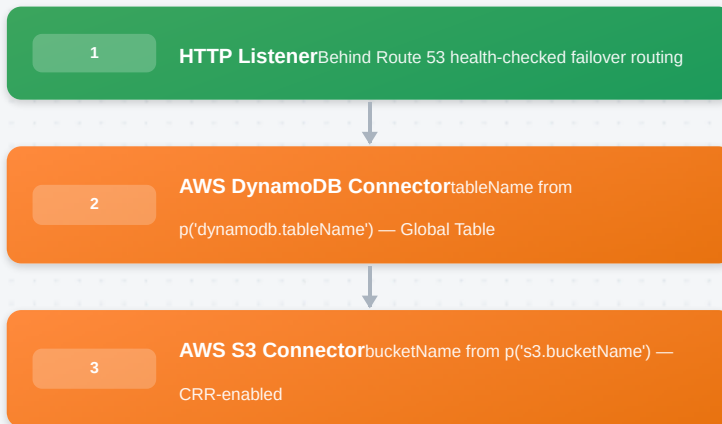
AWS AWS DynamoDB Connector (Global Tables)

AWS AWS S3 Connector (Cross-Region Replication)

STEPS TO CREATE THE MULE FLOW

- 1 Externalize every region-specific value — table name, bucket name, queue URLs — into environment properties, never hardcoded.
- 2 Deploy the identical application to a secondary-region worker, configured with that region's properties.
- 3 Confirm DynamoDB Global Tables and S3 CRR are actively replicating, and Route 53 health checks are configured to redirect on primary failure.

PROCESS-ORDER-FLOW (DEPLOYED IDENTICALLY IN BOTH REGIONS)



MULE CONFIGURATION XML

```

<flow name="process-order-flow">
  <http:listener path="/orders/process" config-ref="HTTP_Listener_config"/>
    <dynamodb:put-item config-ref="DynamoDB_Config" doc:name="Record order (Global
Table)"
      tableName="${dynamodb.tableName}">
      <dynamodb:item>#{[ orderId: { S: vars.orderId }, region: { S: p('aws.region') },
status: { S: 'RECEIVED' } ]}</dynamodb:item>
    </dynamodb:put-item>
    <s3:upload config-ref="S3_Config" doc:name="Archive order (CRR-enabled bucket)"
      bucketName="${s3.bucketName}" key="#[vars.orderId ++ '.json']">
      <s3:content>#[payload]</s3:content>
    </s3:upload>
  </flow>

<!-- us-east-1.properties -->
<!-- aws.region=us-east-1 -->
<!-- dynamodb.tableName=orders-global -->
<!-- s3.bucketName=order-archive-primary -->

<!-- us-west-2.properties -->
<!-- aws.region=us-west-2 -->
<!-- dynamodb.tableName=orders-global -->
<!-- s3.bucketName=order-archive-secondary -->

```

EXPLANATION OF THE CODE

`tableName="${dynamodb.tableName}"` resolving to the same Global Table name in both properties files

A
DynamoDB
Global
Table is a
single
logical table
replicated
across
regions
under the
same name,
so both the
primary and
secondary
deployments
read and
write the
identical
table — a
write

accepted in
us-east-1
becomes
visible in
us-west-2
within the
Global
Table's
typical sub-
second
replication
lag, without
any Mule-
side
replication
logic at all.

`s3.bucketName` differing between the two properties files

Unlike the DynamoDB table, S3 Cross-Region Replication works between two distinctly named buckets rather than one shared logical bucket, so the flow's S3 configuration is deliberately region-specific — each deployment writes to its own regional bucket, and CRR, configured entirely on the S3 side, keeps a replica flowing to the other.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Confirm the flow's DynamoDB and S3 configuration elements reference only property placeholders, never literal region-specific names.
- 2 Create separate properties files per region (`us-east-1.properties`, `us-west-2.properties`) with region-appropriate values.
- 3 Deploy the exact same application package to workers in both regions, each started with its region's properties file.
- 4 Coordinate with the platform team to confirm Global Tables replication, S3 CRR, and Route 53 failover health checks are all active.

TESTING IN POSTMAN

Test each regional deployment independently, then verify Route 53 failover by simulating a health-check failure against the primary.

POST `https://orders.example.com/orders/process`

SAMPLE RESPONSE (AFTER SIMULATED PRIMARY-REGION FAILURE)

```
{ "orderId": "ORD-2201", "servedByRegion": "us-west-2", "status": "RECEIVED" }
```

BRIEFING ALL THE STEPS IN WORDS

The exact same Mule flow deploys to both regions, differing only in which region-specific properties file it starts with, while a DynamoDB Global Table keeps order state consistently replicated between them and S3 Cross-Region Replication does the same for archived documents — so when Route 53's health checks detect the primary region has failed and redirect traffic to the secondary, the flow that picks up the next request already has the state it needs, with no manual rebuild or data migration required at failover time.

SECTION 2 Distributed Patterns & Operational Discipline (Problems 91–100)

#91 Building an End-to-End Data-Lake Ingestion Pipeline (API → S3 → Glue → Redshift)

AWS

PROBLEM STATEMENT

A partner sends daily sales data through an API; that data needs to land in the data lake's raw zone, get curated by Glue, and end up queryable in Redshift — a full pipeline rather than three separately-scheduled pieces that hope to line up correctly.

DETAILED EXPLANATION

This problem is the Volume 5 pattern applied end to end: it chains Problem #84's Glue trigger-and-poll pattern and Problem #83's Redshift COPY pattern into a single orchestrated pipeline, triggered by the API ingestion step itself rather than by three independent schedules. Each stage only starts once the previous one has actually finished, which is what turns three individually correct integrations into one pipeline that's correct as a whole.

CONNECTORS & COMPONENTS USED

CORE

HTTP Listener

AWS

AWS S3 Connector – Upload

AWS

AWS Glue Connector – Start Job Run

AWS

AWS Redshift Data Connector – Execute Statement

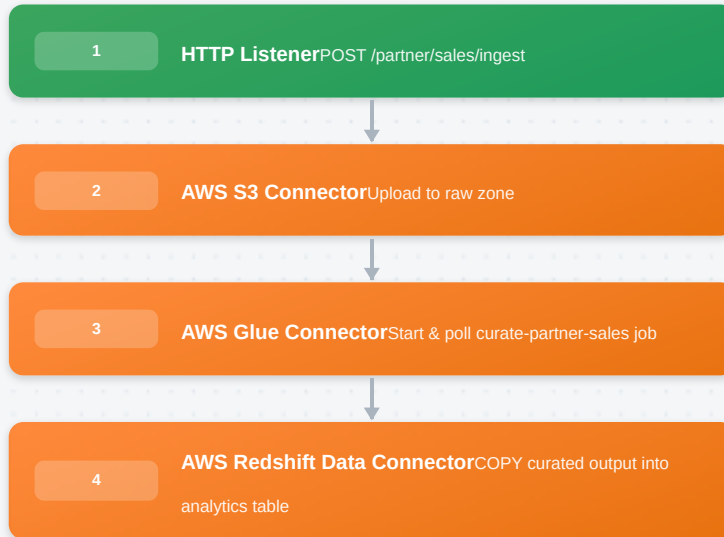
ERR

Until Successful

STEPS TO CREATE THE MULE FLOW

- 1 Accept the partner's daily sales file over HTTP and land it in the raw S3 zone.
- 2 Start the curation Glue job and poll it to completion.
- 3 Once curation succeeds, COPY the curated output into the Redshift analytics table.

INGEST-PARTNER-SALES-PIPELINE-FLOW



MULE CONFIGURATION XML

```
<flow name="ingest-partner-sales-pipeline-flow">
  <http:listener path="/partner/sales/ingest" config-ref="HTTP_Listener_config"/>
  <s3:upload config-ref="S3_Config" doc:name="Land raw sales file"
    bucketName="data-lake-raw" key="#['partner-sales/' ++ (now() as String {format:
'yyyy-MM-dd'}) ++ '.csv'}]">
    <s3:content>#[payload]</s3:content>
  </s3:upload>
  <flow-ref name="run-curate-partner-sales-etl-subflow" doc:name="Run Glue curation
(start + poll)"/>
  <flow-ref name="load-curated-sales-to-redshift-subflow" doc:name="Load curated output
into Redshift"/>
  <set-payload value="#[{ ingested: true, pipeline: 'complete' }]" />
</flow>
```

EXPLANATION OF THE CODE

two flow-ref calls chaining Glue then Redshift, both synchronous

Because each referenced subflow (built following Problems #84 and #83) internally polls its own AWS step to completion before returning, calling them in sequence here is what guarantees the Redshift load never starts against curated data that Glue hasn't actually finished producing yet.

key includes the ingestion date, consistently across the raw file and the later COPY step

Using the same date-derived naming convention at every stage of the pipeline — raw landing, Glue's expected input, and (inside the Redshift subflow) the staged COPY source — is what lets each stage locate the previous stage's output without needing to pass an explicit file path through as a separate parameter.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** and the S3 **Upload** operation to land the raw file.
- 2 Add a **Flow Reference** to a subflow built following Problem #84's Glue start-and-poll pattern.
- 3 Add a second **Flow Reference** to a subflow built following Problem #83's Redshift COPY-and-poll pattern.
- 4 Save and run, then confirm the full chain completes and the data appears in Redshift.

TESTING IN POSTMAN

POST

https://localhost:8081/partner/sales/ingest

Header

Value

Content-Type

text/csv

SAMPLE RESPONSE

JSON

```
{ "ingested": true, "pipeline": "complete" }
```

BRIEFING ALL THE STEPS IN WORDS

The S3 Connector's Upload operation lands the partner's daily sales file in the data lake's raw zone, a Flow Reference to a Glue start-and-poll subflow curates it, and a second Flow Reference to a Redshift COPY subflow loads the curated result into the analytics table — chaining three previously individual integrations into one pipeline where each stage's completion is what triggers the next, rather than three independent schedules hoping to line up.

#92 Correlating Logs and Traces Across a MuleSoft + AWS Microservices Architecture

AWS

PROBLEM STATEMENT

A single customer request now touches a Mule gateway, two Lambda functions, and an ECS service. When something goes wrong, engineers need one search that surfaces every log line and trace segment from that specific request, not four separate systems to check by hand.

DETAILED EXPLANATION

This builds directly on Problem #74's X-Ray trace propagation: the same trace ID that ties X-Ray segments together is also the key that makes cross-system log correlation possible, as long as every service — Mule included — includes it in every structured log line it writes. With that discipline in place consistently, a CloudWatch Logs Insights query filtering on one traceId value returns every log line from every hop of that one request, and the X-Ray console shows the same request's full latency breakdown, giving engineers two complementary views keyed by the exact same identifier.

CONNECTORS & COMPONENTS USED

AWS

AWS CloudWatch Logs Connector – Put Log Events

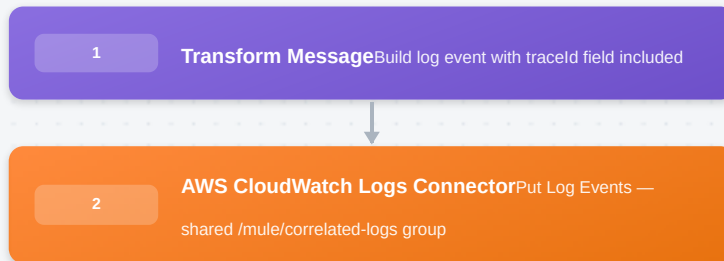
CORE

Global Logger Configuration

STEPS TO CREATE THE MULE FLOW

- 1 Resolve or propagate the trace ID exactly as in Problem #74.
- 2 Include the trace ID as a field in every structured log event this flow (and all its subflows) writes.
- 3 Configure the same trace-ID field name and format in every other service's own logging so it lines up in Logs Insights.

CORRELATED-LOG-HELPER-SUBFLOW



MULE CONFIGURATION XML

```
<sub-flow name="correlated-log-helper-subflow">
  <ee:transform doc:name="Build correlated log event">
    <ee:variables>
      <ee:set-variable variableName="logEvent">#[%dw 2.0
output application/json
---
{
  traceId: vars.traceId,
  service: "mule-order-gateway",
  correlationId: correlationId,
  level: vars.logLevel default "INFO",
  message: vars.logMessage
}]]</ee:set-variable>
    </ee:variables>
  </ee:transform>
  <cloudwatch-logs:put-log-events config-ref="CloudwatchLogs_Config" doc:name="Write
correlated log event"
  logGroupName="/mule/correlated-logs" logStreamName="#"['worker-' ++
p('worker.id')]"]>
    <cloudwatch-logs:log-events>#[[ { timestamp: now() as Number, message:
write(vars.logEvent, "application/json") } ]]]</cloudwatch-logs:log-events>
  </cloudwatch-logs:put-log-events>
</sub-flow>
```

EXPLANATION OF THE CODE

service: "mule-order-gateway" as a fixed field

Tagging every log event with a constant service name is what makes a Logs Insights query across the shared log group able to distinguish which hop of the request each line came from, once the Lambda functions and the ECS service are logging into the same group with their own service names alongside the same traceId.

this subflow referenced from every flow that logs, rather than duplicated

Centralizing the log-event shape and the CloudWatch call in one reusable subflow is what keeps the `traceId` field, its name, and its format consistent everywhere it's used — a single flow forgetting to include it, or spelling it differently, would silently break correlation for every request that flow handles.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Create a reusable `Sub-Flow` named `correlated-log-helper-subflow` with a Transform Message building the standard log event shape.
- 2 Add the CloudWatch Logs `Put Log Events` operation writing to a shared `/mule/correlated-logs` group.
- 3 Reference this subflow with `Flow Reference` anywhere a flow needs to log, setting `logMessage` and `logLevel` variables beforehand.
- 4 Confirm the Lambda and ECS teams configure their own logging to include the same `traceId` field name, then save and run.

TESTING IN POSTMAN

POST `https://localhost:8081/orders/process`

SAMPLE CLOUDWATCH LOGS INSIGHTS QUERY

```
fields @timestamp, service, message | filter traceId = "Root=1-63a1b2c3-abcdef1234567890abcdef12" | sort @timestamp asc
```

BRIEFING ALL THE STEPS IN WORDS

A shared logging subflow builds a structured event carrying the same trace ID propagated through X-Ray in Problem #74, tagged with a fixed service name, and the CloudWatch Logs Connector's Put Log Events operation writes it into a log group shared across every service in the architecture — so one Logs Insights query filtering on a single trace ID surfaces the complete, chronologically ordered story of one request across the Mule gateway, both Lambda functions, and the ECS service, instead of four separate places to look.

#93 Implementing a Saga Pattern for a Distributed Transaction Spanning Mule and AWS

AWS

PROBLEM STATEMENT

Booking a travel package reserves a flight seat via one Lambda, a hotel room via DynamoDB, and a rental car via another Lambda — three independent systems with no shared transaction. If the car reservation fails after the flight and hotel succeeded, both of those need to be explicitly released, not left dangling.

DETAILED EXPLANATION

A distributed transaction across independent AWS services can't use a database-style atomic commit, so the saga pattern handles it instead: each step has a matching compensating action that undoes it, and if any step in the sequence fails, the flow runs the compensating actions for every step that already succeeded, in reverse order. This flow tracks which steps have completed in a variable as it goes, so the compensation logic knows exactly what needs undoing rather than blindly attempting to reverse steps that never actually ran.

CONNECTORS & COMPONENTS USED

AWS

AWS Lambda Connector – Invoke (flight, car)

AWS

AWS DynamoDB Connector – Put Item / Delete Item (hotel)

ERR

Try / On Error Propagate

STEPS TO CREATE THE MULE FLOW

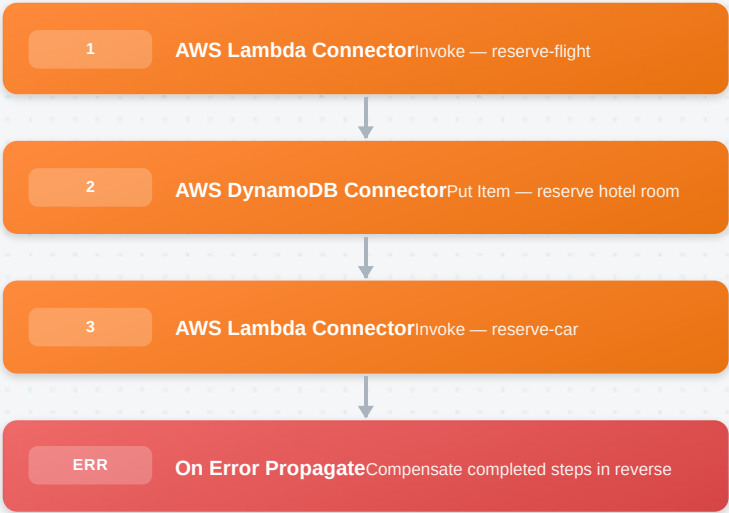
- 1

Reserve the flight, then the hotel, then the car, tracking each success in a completedSteps variable.
- 2

On any step's failure, catch the error before it propagates further.
- 3

Run compensating actions for every completed step, in reverse order, then report the booking as failed.

BOOK-TRAVEL-PACKAGE-SAGA-FLOW



MULE CONFIGURATION XML

```

<flow name="book-travel-package-saga-flow">
  <set-variable variableName="completedSteps" value="#[[]]"/>
  <try>
    <lambda:invoke config-ref="Lambda_Config" doc:name="Reserve flight"
      functionName="reserve-flight" invocationType="RequestResponse"
      target="flightResult"/>
    <set-variable variableName="completedSteps" value="#[vars.completedSteps ++
['flight']]"/>

    <dynamodb:put-item config-ref="DynamoDB_Config" doc:name="Reserve hotel room"
      tableName="hotel-reservations">
      <dynamodb:item>#{[ reservationId: { S: vars.bookingId }, status: { S: 'RESERVED'
} ]}</dynamodb:item>
    </dynamodb:put-item>
    <set-variable variableName="completedSteps" value="#[vars.completedSteps ++
['hotel']]"/>

    <lambda:invoke config-ref="Lambda_Config" doc:name="Reserve car"
      functionName="reserve-car" invocationType="RequestResponse"
      target="carResult"/>
    <set-variable variableName="completedSteps" value="#[vars.completedSteps ++
['car']]"/>

    <error-handler>
      <on-error-propagate type="ANY">
        <foreach collection="#[vars.completedSteps reverse]" doc:name="Compensate
completed steps in reverse">
          <choice>
            <when expression="#[payload == 'car']">
              <lambda:invoke config-ref="Lambda_Config" doc:name="Cancel car
reservation"
                functionName="cancel-car" invocationType="RequestResponse"/>
            </when>
            <when expression="#[payload == 'hotel']">
              <dynamodb:delete-item config-ref="DynamoDB_Config" doc:name="Cancel hotel
reservation"
                tableName="hotel-reservations" key="#[{ reservationId: { S:
vars.bookingId } } ]"/>
            </when>
            <when expression="#[payload == 'flight']">
              <lambda:invoke config-ref="Lambda_Config" doc:name="Cancel flight
reservation"
                functionName="cancel-flight" invocationType="RequestResponse"/>
            </when>
          </choice>
        </foreach>
      </on-error-propagate>
    </error-handler>
  </try>
</flow>

```

```

    </foreach>
    <logger level="ERROR" message="Travel package booking failed – all completed
steps compensated"/>
  </on-error-propagate>
</error-handler>
</try>
</flow>

```

EXPLANATION OF THE CODE

`vars.completedSteps ++ ['flight']` appended right after each step succeeds

Recording each step's completion immediately after it succeeds, rather than assuming all prior steps ran, means the compensation loop only ever attempts to undo work that genuinely happened — if the car reservation is the one that fails, `completedSteps` contains exactly `['flight', 'hotel']` and nothing more.

`vars.completedSteps reverse` inside the `foreach`

Compensating in the reverse of the original order matters when later reservations could plausibly depend on earlier ones still being held — canceling the most recently completed step first and working backward is the safer default even when, as here, the three reservations are largely independent of each other.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Add a `Set Variable` initializing `completedSteps` to an empty array, then wrap the three reservation steps in a `Try` scope.
- 2 After each reservation operation, add a `Set Variable` appending that step's name to `completedSteps`.
- 3 Add an `On Error Propagate` handler containing a `For Each` over the reversed `completedSteps` list.
- 4 Inside the loop, add a `Choice` router calling the matching compensating action for each step name, then save and run.

TESTING IN POSTMAN

POST `https://localhost:8081/travel/book-package`

SAMPLE RESPONSE (CAR RESERVATION FAILED)

 JSON

```
{ "booked": false, "compensated": ["car", "hotel", "flight"] }
```

BRIEFING ALL THE STEPS IN WORDS

Each reservation step — the Lambda Connector's Invoke operation for flight and car, the DynamoDB Connector's Put Item operation for hotel — records its own success into a completedSteps list as it runs, and when any step fails, an On Error Propagate handler walks that list in reverse, calling the matching cancellation action for exactly the steps that actually completed — leaving no dangling flight, hotel, or car reservation behind when a booking can't be completed as a whole.

#94 Building a Real-Time Inventory-Sync Pipeline Using DynamoDB Streams and Mule

AWS

PROBLEM STATEMENT

Whenever a warehouse system updates a product's quantity in DynamoDB, the storefront's search index needs to reflect that change within a few seconds — not on the next scheduled sync, and not by having the warehouse system call the search index directly and take on that coupling.

DETAILED EXPLANATION

DynamoDB Streams captures an ordered, near-real-time log of every item-level change in a table, and Mule can consume that stream directly rather than polling the table for changes. This decouples the warehouse system, which just writes to DynamoDB as it always has, from the search index, which stays current by reacting to the stream — if a new consumer of the same change feed shows up later, it just adds another stream reader, with no change to the warehouse system at all.

CONNECTORS & COMPONENTS USED

AWS

AWS DynamoDB Streams Connector – On New Record

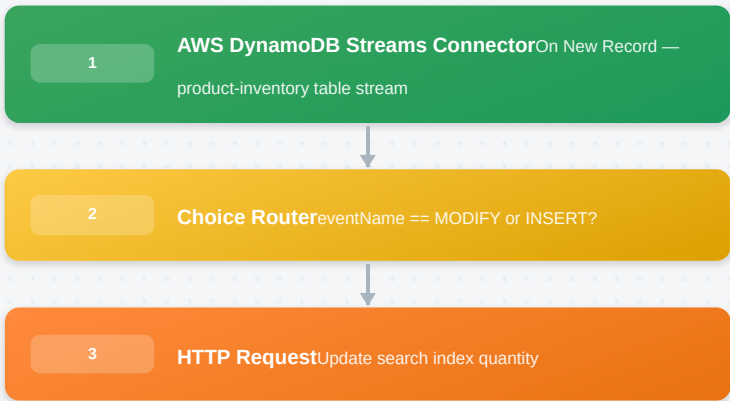
CORE

HTTP Request (search index API)

STEPS TO CREATE THE MULE FLOW

- 1 Listen on the product-inventory table's DynamoDB Stream for new item-change records.
- 2 Filter for MODIFY and INSERT events (ignore REMOVE for this use case) and extract the new quantity.
- 3 Push the updated quantity to the search index's update API.

SYNC-INVENTORY-TO-SEARCH-INDEX-FLOW




```

<flow name="sync-inventory-to-search-index-flow">
  <dynamodb-streams:listener config-ref="DynamoDBStreams_Config" doc:name="On inventory
table change"
    streamArn="${dynamodb.inventoryStreamArn}" shardIteratorType="LATEST"/>
  <choice doc:name="Relevant event type?">
    <when expression="#[(['MODIFY', 'INSERT']) contains attributes.eventName]">
      <ee:transform doc:name="Extract SKU and quantity">
        <ee:variables>
          <ee:set-variable variableName="sku">#[ (payload.dynamodb.newImage.sku).s ]
        </ee:set-variable>
          <ee:set-variable variableName="quantity">#
[ (payload.dynamodb.newImage.quantity).n as Number ]</ee:set-variable>
        </ee:variables>
      </ee:transform>
      <http:request config-ref="SearchIndex_config" method="PATCH" path="/products/#
[vars.sku]" doc:name="Update search index">
        <http:body>#[ { quantity: vars.quantity } ]</http:body>
      </http:request>
    </when>
  </choice>
</flow>

```

EXPLANATION OF THE CODE

`shardIteratorType="LATEST"`

Starting from LATEST means this listener only sees changes that happen from the moment it starts consuming onward — appropriate for a live sync feed, as opposed to TRIM_HORIZON, which would replay the stream's entire retained history and re-sync every historical change on startup.

`(payload.dynamodb.newImage.sku).s`

A DynamoDB Stream record represents each attribute using the same typed-value wrapper as the table itself — a string attribute's value sits under a lowercase s key, a number under n — which is why extracting the SKU and quantity here looks structurally similar to unwrapping a normal DynamoDB item.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag the `AWS DynamoDB Streams` `On New Record` source and set `Stream Arn` and `Shard Iterator Type` to LATEST.
- 2 Add a `Choice` router filtering on the event's eventName attribute.
- 3 Add a `Transform Message` extracting the SKU and quantity from the stream record's new image.
- 4 Drag an `HTTP Request` to the search index's update endpoint, then save and run.

TESTING IN POSTMAN

Stream-triggered. Update an item's quantity in the DynamoDB table directly and confirm the search index reflects it within seconds.

POST `https://localhost:8081/admin/simulate-inventory-change`

SAMPLE RESPONSE

JSON

```
{ "sku": "SKU-4471", "quantity": 132, "indexUpdated": true }
```

BRIEFING ALL THE STEPS IN WORDS

The DynamoDB Streams Connector's On New Record source picks up every item-level change to the inventory table in near real time, a Choice router filters for the change types that actually matter for search, and the HTTP Request operation pushes the updated quantity to the search index's API — keeping the storefront's search results current within seconds of a warehouse update, with the warehouse system itself never needing to know the search index exists.

#95

Designing an Event-Replay and Recovery Strategy Using S3 as a Durable Event Store

AWS

PROBLEM STATEMENT

A bug in the order-enrichment Lambda silently corrupted enrichment data for six hours before anyone noticed. Once fixed, the team needs to reprocess every order event from that window — but SQS messages are long gone by the time the bug is caught, since they're deleted once successfully processed.

DETAILED EXPLANATION

SQS is a transport, not an archive — it's the wrong place to look for something to replay after the fact. This flow solves that by durably archiving a copy of every event to S3, partitioned by date and hour, at the moment it's first received, independent of whatever happens during processing. When a bug like this is discovered, a separate replay flow reads the archived events for the affected window directly from S3 and republishes them through the same processing path, this time with the fix in place.

CONNECTORS & COMPONENTS USED

AWS AWS S3 Connector – Upload (archive) / List Objects / Get Object (replay)

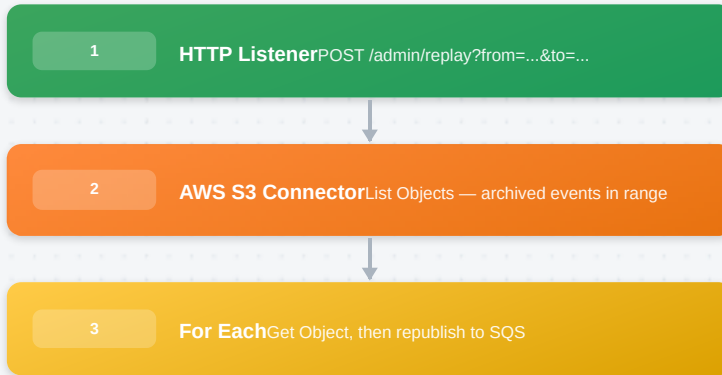
AWS AWS SQS Connector – Publish (replay)

STEPS TO CREATE THE MULE FLOW

- 1 On every order event received, archive a copy to S3 under an hour-partitioned key, before processing continues.
- 2 Build a separate, manually-triggered replay flow that lists archived events for a given date-hour range.

- 3 Republish each archived event's content back onto the processing queue.

REPLAY-ARCHIVED-ORDER-EVENTS-FLOW



MULE CONFIGURATION XML

```
<flow name="replay-archived-order-events-flow">
  <http:listener path="/admin/replay" config-ref="HTTP_Listener_config"/>
  <s3:list-objects config-ref="S3_Config" doc:name="List archived events in range"
    bucketName="order-event-archive"
    prefix="#['events/' ++ attributes.queryParams.from]" target="archivedEvents"/>
  <foreach collection="#[vars.archivedEvents.objectSummaries]" doc:name="Replay each archived event">
    <s3:get-object config-ref="S3_Config" doc:name="Fetch archived event"
      bucketName="order-event-archive" key="#[payload.key]" target="archivedEvent"/>
    <sqs:publish config-ref="SQS_Config" doc:name="Republish to processing queue"
      queueUrl="${sqs.orderCreatedQueueUrl}" content="#[vars.archivedEvent.content]">
      <sqs:message-attributes>#{[ replay: { dataType: 'String', stringValue: 'true' }
    ]}</sqs:message-attributes>
    </sqs:publish>
  </foreach>
  <logger level="INFO" message="#['Replayed ' ++
    (sizeof(vars.archivedEvents.objectSummaries) as String) ++ ' archived events']"/>
</flow>
```

EXPLANATION OF THE CODE

prefix built from the from query parameter, e.g. events/2026-08-27/14

Partitioning the archive by date and hour in the object key itself is what makes a targeted replay practical — listing objects under a specific hour's prefix returns exactly the affected window's events without needing to

scan or filter the entire archive by timestamp.

```
replay: { dataType: 'String', stringValue: 'true' } message attribute
```

Tagging a replayed message so the downstream processing flow can distinguish it from a fresh, first-time delivery is useful for observability — a spike in processed volume that's actually a deliberate replay looks very different from one caused by a genuine traffic surge, and this attribute is what makes that distinction visible in logs and metrics.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 In the main order-ingestion flow, add an S3 **Upload** operation archiving every event under an hour-partitioned key before processing continues.
- 2 Build a separate admin-only flow with an **HTTP Listener** and the S3 **List Objects** operation scoped by the requested date-hour prefix.
- 3 Add a **For Each** loop with S3 **Get Object** and SQS **Publish** operations to republish each archived event.
- 4 Restrict this admin endpoint to internal access only, then save and run.

TESTING IN POSTMAN

POST `https://localhost:8081/admin/replay?from=events/2026-08-27/14`

SAMPLE RESPONSE

JSON

```
{ "eventsReplayed": 214, "window": "2026-08-27T14:00-15:00" }
```

BRIEFING ALL THE STEPS IN WORDS

Every order event gets archived to a date-and-hour-partitioned S3 key at the moment it's first received, independent of processing outcome, and a separate admin-triggered replay flow uses the S3 Connector's List Objects and Get Object operations to retrieve exactly the affected window's events and the SQS Connector's Publish operation to republish them — giving the team a real recovery path for a corruption bug discovered hours after the fact, something the transient SQS queue alone could never provide.

Building a Cost-Aware AWS Integration That Batches Calls to Reduce Lambda Invocations

AWS

PROBLEM STATEMENT

A click-tracking flow invokes a Lambda function once per event to record analytics, and at current volume — around 2 million events a day — the per-invocation Lambda cost has become a noticeable line item. The team wants to batch events together to cut the invocation count without losing near-real-time processing.

DETAILED EXPLANATION

Lambda bills per invocation and per unit of compute time, so calling it once per event scales invocation count linearly with traffic even though a single invocation could easily process dozens of events in about the same wall-clock time. This flow buffers incoming events in an SQS queue and processes them in batches by configuring the SQS listener to fetch up to ten messages per poll, cutting the Lambda invocation count roughly by that same factor while keeping end-to-end latency to a few seconds at most.

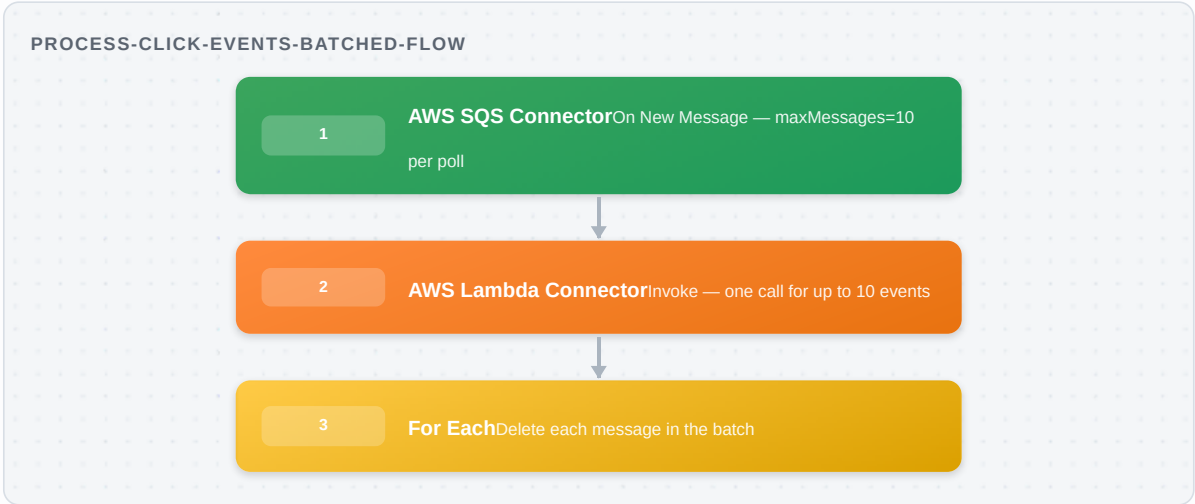
CONNECTORS & COMPONENTS USED

AWS AWS SQS Connector – On New Message (batched)

AWS AWS Lambda Connector – Invoke (batch payload)

STEPS TO CREATE THE MULE FLOW

- 1 Configure the SQS listener to fetch up to 10 messages per poll instead of 1.
- 2 Group the polled messages into a single array payload for one Lambda call.
- 3 Invoke Lambda once with the whole batch and delete all successfully-processed messages together.



MULE CONFIGURATION XML

```

<flow name="process-click-events-batched-flow">
  <sqs:listener config-ref="SQS_Config" queueUrl="${sqs.clickEventsQueueUrl}"
    ackMode="MANUAL" maxMessages="10" primaryNodeOnly="false"/>
  <lambda:invoke config-ref="Lambda_Config" doc:name="Record click batch"
    functionName="record-click-batch" invocationType="RequestResponse"
    target="batchResult">
    <lambda:payload>#[%dw 2.0
output application/json
---
{ events: payload map (m) -> read(m.body, "application/json") }]</lambda:payload>
  </lambda:invoke>
  <foreach collection="#[payload]" doc:name="Delete each processed message">
    <sqs:delete-message config-ref="SQS_Config" queueUrl="${sqs.clickEventsQueueUrl}"
      receiptHandle="#[payload.attributes.receiptHandle]"/>
  </foreach>
</flow>

```

EXPLANATION OF THE CODE

maxMessages="10" on sqs:listener

Raising the listener's batch fetch size from SQS's default single-message poll to ten is what actually reduces the invocation count — one flow execution now represents up to ten events instead of one, and the Lambda call downstream naturally follows that same batching.

{ events: payload map (m) -> read(m.body, "application/json") }

Building a single array of all ten events' parsed bodies and sending it as one Lambda payload is what lets record-click-batch process the whole group inside one invocation, rather than the Mule flow still calling Lambda once per message despite having fetched them in a batch.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 On the SQS **On New Message** source, set **Max Messages** to 10.
- 2 Add **AWS Lambda > Invoke** with a Payload expression mapping the whole batch into one events array.
- 3 Add a **For Each** over the original batch, with an SQS **Delete Message** operation inside deleting each one by its receipt handle.
- 4 Save and run, then compare Lambda's invocation count against the previous per-event implementation.

TESTING IN POSTMAN

SQS-triggered. Publish several click events in quick succession and confirm one Lambda invocation handles them together.

POST

https://localhost:8081/admin/simulate-click-batch

SAMPLE RESPONSE

JSON

```
{ "eventsBatched": 10, "lambdaInvocations": 1 }
```

BRIEFING ALL THE STEPS IN WORDS

The SQS Connector's On New Message source polls up to ten messages at once instead of one, the Lambda Connector's Invoke operation processes that whole batch in a single call by sending all ten events as one payload, and a For Each loop deletes each message individually once the batch succeeds — cutting Lambda invocation count roughly tenfold at the current volume while keeping processing latency within a few seconds of real time.

#97

Documenting Blue-Green Deployment Considerations for a MuleSoft-AWS Integration App

AWS

PROBLEM STATEMENT

The team wants zero-downtime deployments for the order-processing integration, but a naive blue-green cutover risks in-flight SQS messages and DynamoDB writes being processed by both the old and new version simultaneously during the switch, or lost entirely.

DETAILED EXPLANATION

Blue-green deployment for a stateless HTTP API is comparatively simple — route traffic to the new version once it's healthy, done. An event-driven integration consuming from SQS is trickier, because both the old (blue) and new (green) worker could be actively polling and processing from the same queue during the overlap window, which is actually fine for most stateless processing but becomes a correctness risk if the new version's logic isn't backward-compatible with in-flight message formats. The safe pattern documented here: deploy green alongside blue with both consuming the same queue, verify green's error rate and behavior on live traffic for a defined soak period, then scale blue down to zero — never a hard cutover that could drop messages mid-poll.

CONNECTORS & COMPONENTS USED

AWS

AWS SQS Connector – On New Message (both versions, shared queue)

AWS

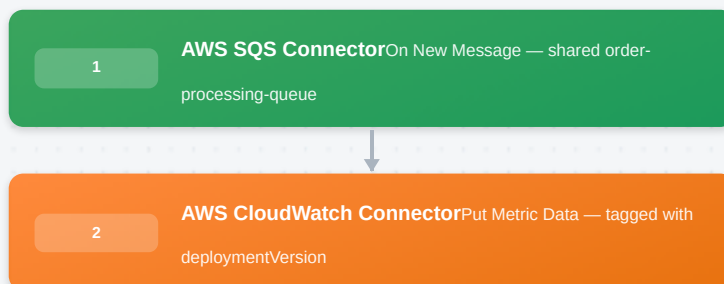
AWS CloudWatch Connector – Put Metric Data (per-version error rate)

STEPS TO CREATE THE MULE FLOW

- 1 Deploy the green version alongside blue, both polling the same SQS queue, with message schema changes kept backward-compatible.

- 2 Tag every published metric with the deployment version, so blue and green's error rates can be compared side by side.
- 3 Once green's metrics are healthy over the soak period, scale blue's worker count to zero.

PROCESS-ORDER-FLOW (DEPLOYED AS BOTH BLUE AND GREEN)



MULE CONFIGURATION XML

```
<flow name="process-order-flow">
  <sqs:listener config-ref="SQS_Config" queueUrl="${sqs.orderProcessingQueueUrl}"
  ackMode="MANUAL"/>
  <try>
    <flow-ref name="process-order-subflow"/>
    <cloudwatch:put-metric-data config-ref="CloudWatch_Config" doc:name="Report
  success" namespace="Integration/OrderProcessing">
      <cloudwatch:metric-data>#[ [ { metricName: "ProcessedOK", value: 1, unit: "Count",
        dimensions: [ { name: "deploymentVersion", value: p('deployment.version') } ]
      } ] ]</cloudwatch:metric-data>
    </cloudwatch:put-metric-data>
    <error-handler>
      <on-error-continue type="ANY">
        <cloudwatch:put-metric-data config-ref="CloudWatch_Config" doc:name="Report
  failure" namespace="Integration/OrderProcessing">
            <cloudwatch:metric-data>#[ [ { metricName: "ProcessedError", value: 1, unit:
  "Count",
              dimensions: [ { name: "deploymentVersion", value: p('deployment.version')
            } ] } ] ]</cloudwatch:metric-data>
          </cloudwatch:put-metric-data>
        </on-error-continue>
      </error-handler>
    </try>
  </flow>
```

EXPLANATION OF THE CODE


```
dimensions: [ { name: "deploymentVersion", value: p('deployment.version') } ]
```

Both blue and green are the same flow definition, deployed twice with different deployment.version property values — tagging every metric with that dimension is what lets a single CloudWatch dashboard split ProcessedOK and ProcessedError side by side per version during the soak period, which is the actual decision data for when it's safe to scale blue down.

both blue and green listening on the exact same queueUrl

SQS distributes messages across however many consumers are polling a queue, so having both versions active simultaneously naturally splits live traffic between them without any explicit routing rule — each message is still processed exactly once, just by whichever worker happens to poll it first.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Confirm the flow's SQS listener configuration references the shared queue URL via a property, unchanged between versions.
- 2 Add success and failure `CloudWatch > Put Metric Data` operations, both tagged with a deploymentVersion dimension from a property.
- 3 Deploy the new version as a separate worker (green) alongside the existing one (blue), each with its own deployment.version property value.
- 4 Watch the CloudWatch dashboard split by version during the soak period, then scale blue's worker count to zero once green is confirmed healthy.

TESTING IN POSTMAN

SQS-triggered on both deployed versions simultaneously; verify via the CloudWatch dashboard rather than a single request/response.

GET CloudWatch console → Integration/OrderProcessing → ProcessedOK by deploymentVersion

SAMPLE METRIC SNAPSHOT

JSON

```
{ "blue": { "processedOK": 812, "processedError": 1 }, "green": { "processedOK": 794, "processedError": 0 } }
```

BRIEFING ALL THE STEPS IN WORDS

Both the blue and green deployments run the exact same flow definition polling the exact same SQS queue, letting SQS split live traffic between them naturally, while the CloudWatch Connector's Put Metric Data operation tags every success and failure with which version processed it — giving the team the side-by-side error-rate comparison that determines when it's actually safe to scale the old version down, instead of a hard cutover that risks in-flight messages being dropped or double-processed.

#98

Building a Monitoring-Dashboard Data Feed Combining Mule and CloudWatch Metrics

AWS

PROBLEM STATEMENT

Operations wants a single internal dashboard showing both Mule-side metrics (orders processed, error rate) and native AWS metrics (Lambda duration, SQS queue depth) for the order-processing integration, refreshed every minute, rather than switching between Anypoint Monitoring and the CloudWatch console.

DETAILED EXPLANATION

Rather than building a new dashboarding tool, this flow acts as a lightweight aggregation API: it queries CloudWatch's GetMetricData for the relevant AWS-side metrics, combines them with counts pulled from the app's own DynamoDB-based metrics store, and returns one unified JSON payload the actual dashboard's frontend polls on a timer. This keeps CloudWatch as the source of truth for AWS metrics rather than duplicating that data, while still presenting everything through one API a simple dashboard can consume.

CONNECTORS & COMPONENTS USED

CORE HTTP Listener

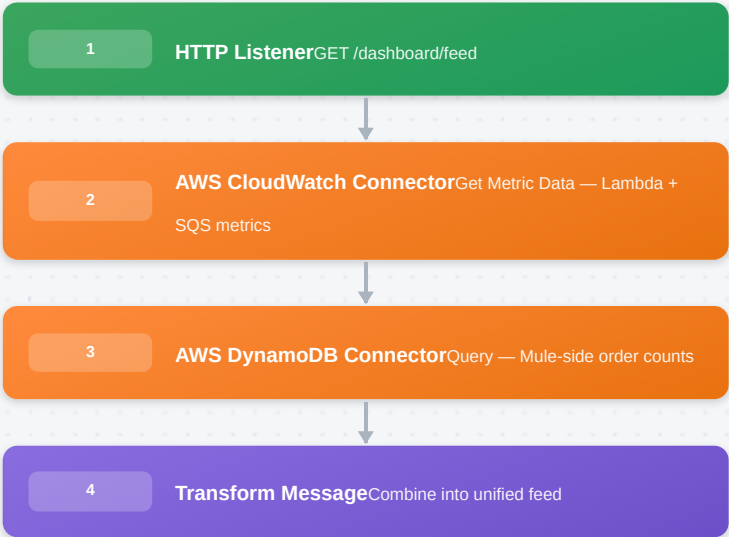
AWS AWS CloudWatch Connector – Get Metric Data

AWS AWS DynamoDB Connector – Query

STEPS TO CREATE THE MULE FLOW

- 1 Query CloudWatch for Lambda duration and SQS queue-depth metrics over the last 15 minutes.
- 2 Query the app's own DynamoDB metrics store for orders-processed and error counts over the same window.
- 3 Combine both into a single JSON response for the dashboard to poll.

GET-DASHBOARD-FEED-FLOW



MULE CONFIGURATION XML

```

<flow name="get-dashboard-feed-flow">
  <http:listener path="/dashboard/feed" config-ref="HTTP_Listener_config"/>
  <cloudwatch:get-metric-data config-ref="CloudWatch_Config" doc:name="Fetch AWS-side
metrics"
    startTime="#[now() - |PT15M|]" endTime="#[now()]" target="awsMetrics">
    <cloudwatch:metric-data-queries>#[[
      { id: "lambdaDuration", metricStat: { metric: { namespace: "AWS/Lambda", metricName:
"Duration",
        dimensions: [{ name: "FunctionName", value: "score-batch-record" }] }, period:
60, stat: "Average" } },
      { id: "queueDepth", metricStat: { metric: { namespace: "AWS/SQS", metricName:
"ApproximateNumberOfMessagesVisible",
        dimensions: [{ name: "QueueName", value: "order-processing-queue" }] }, period:
60, stat: "Maximum" } }
    ]]</cloudwatch:metric-data-queries>
    </cloudwatch:get-metric-data>
    <dynamodb:query config-ref="DynamoDB_Config" doc:name="Fetch Mule-side counts"
tableName="integration-metrics"
      keyConditionExpression="metricWindow = :w" target="muleMetrics">
      <dynamodb:expression-attribute-values>#[[ { ':w': { S: now() as String {format:
"yyyy-MM-dd'T'HH:mm"} } } ]]</dynamodb:expression-attribute-values>
    </dynamodb:query>
    <ee:transform doc:name="Build unified feed">
      <ee:message>
        <ee:set-payload>#[%dw 2.0
output application/json
---
{
  lambdaAvgDurationMs: (vars.awsMetrics.metricDataResults[0]).values[0] default 0,
  sqsQueueDepth: (vars.awsMetrics.metricDataResults[1]).values[0] default 0,
  ordersProcessed: sizeof(vars.muleMetrics.items),
  generatedAt: now() as String
}]</ee:set-payload>
      </ee:message>
    </ee:transform>
  </flow>

```

EXPLANATION OF THE CODE

two entries in `cloudwatch:metric-data-queries`, each with its own id

Get Metric Data supports batching several distinct metric queries into one API call, and the id field on each is what lets the response's `metricDataResults` array be matched back to the right query — the first entry's values correspond to Lambda

dynamodb:query keyed on the current minute-granularity metricWindow

duration, the second's to SQS queue depth, in the same order they were requested.

Storing Mule-side counts pre-aggregated by minute in DynamoDB, rather than querying raw event records on every dashboard poll, keeps this endpoint fast even as historical order volume grows — the aggregation work happens once, at write time, not on every read.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an HTTP Listener for /dashboard/feed.
- 2 Add AWS CloudWatch > Get Metric Data with a Metric Data Queries expression covering both AWS-side metrics.
- 3 Add the DynamoDB Query operation against the pre-aggregated integration-metrics table.
- 4 Add a Transform Message combining both results into one JSON response, then save and run.

TESTING IN POSTMAN

GET https://localhost:8081/dashboard/feed

SAMPLE RESPONSE

JSON

{ "lambdaAvgDurationMs": 214.5, "sqsQueueDepth": 12, "ordersProcessed": 341, "generatedAt": "2026-08-28T10:15:00Z" }

BRIEFING ALL THE STEPS IN WORDS

The CloudWatch Connector's Get Metric Data operation retrieves Lambda duration and SQS queue-depth metrics in one batched call, the DynamoDB Connector's Query operation retrieves pre-aggregated Mule-side order counts for the same window, and a transform combines both into one unified JSON feed — letting a simple internal dashboard poll a single endpoint instead of stitching together Anypoint Monitoring and the CloudWatch console by hand.

#99

Designing a Disaster-Recovery Runbook That Fails MuleSoft Flows Over to a Backup AWS Region

AWS
PROBLEM STATEMENT

Problem #90 built the technical foundation for multi-region failover; this problem is about the operational runbook that turns that foundation into something a human can actually execute correctly at 3am during a real regional outage, under pressure, without missing a step.

DETAILED EXPLANATION

A DR runbook's value is in being concrete and mechanically checkable, not in describing the architecture again — it should be a short, ordered sequence of verification and action steps, each with a way to confirm it actually worked, that an on-call engineer with no deep AWS expertise can follow. This flow builds a single admin endpoint that automates the verification half of that runbook — checking Global Table replication lag, S3 CRR status, and secondary-region worker health all at once — so the human executing the failover gets a clear go/no-go signal instead of manually checking three AWS consoles under pressure.

CONNECTORS & COMPONENTS USED

AWS

AWS DynamoDB Connector – Describe Table (replication status)

AWS

AWS CloudWatch Connector – Get Metric Data (replication lag)

CORE

HTTP Request (secondary-region health check)

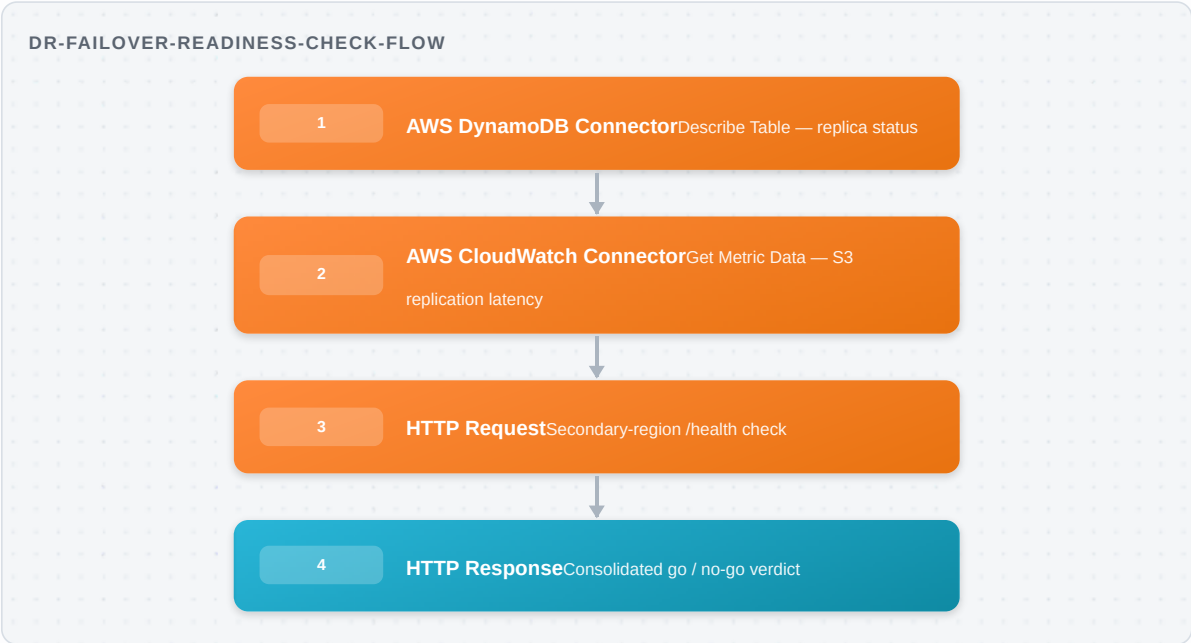
STEPS TO CREATE THE MULE FLOW

- 1

Check the DynamoDB Global Table's replication status for the secondary region.
- 2

Check the S3 Cross-Region Replication metric for pending-replication bytes.
- 3

Ping the secondary-region worker's health endpoint and return one consolidated go/no-go verdict.



MULE CONFIGURATION XML

```

<flow name="dr-failover-readiness-check-flow">
  <http:listener path="/admin/dr-readiness" config-ref="HTTP_Listener_config"/>
  <dynamodb:describe-table config-ref="DynamoDB_Config" doc:name="Check Global Table
replica status"
    tableName="orders-global" target="tableInfo"/>
  <cloudwatch:get-metric-data config-ref="CloudWatch_Config" doc:name="Check S3
replication latency"
    startTime="#[now() - |PT10M|]" endTime="#[now()]" target="crrMetrics">
    <cloudwatch:metric-data-queries>#[[ { id: "replicationLatency", metricStat: {
metric: { namespace: "AWS/S3",
    metricName: "ReplicationLatency", dimensions: [{ name: "SourceBucket", value:
"order-archive-primary" } ] },
    period: 300, stat: "Maximum" } } ]]</cloudwatch:metric-data-queries>
  </cloudwatch:get-metric-data>
  <http:request config-ref="Secondary_Region_config" method="GET" path="/health"
doc:name="Ping secondary-region worker" target="secondaryHealth"/>
  <ee:transform doc:name="Build go/no-go verdict">
    <ee:message>
      <ee:set-payload>#[%dw 2.0
output application/json
var replicaHealthy = (vars.tableInfo.table.replicas[0]).replicaStatus == "ACTIVE"
var replicationLagOk = ((vars.crrMetrics.metricDataResults[0]).values[0] default 999) <
60
var secondaryUp = vars.secondaryHealth.statusCode == 200
---
{
  goForFailover: replicaHealthy and replicationLagOk and secondaryUp,
  replicaHealthy: replicaHealthy,
  replicationLagSeconds: (vars.crrMetrics.metricDataResults[0]).values[0] default 999,
  secondaryRegionUp: secondaryUp
}]]</ee:set-payload>
    </ee:message>
  </ee:transform>
</flow>

```

EXPLANATION OF THE CODE

goForFailover: replicaHealthy and replicationLagOk and secondaryUp

Combining all three checks into one boolean is the entire point of automating this step — the on-call engineer executing the runbook needs one clear answer, not three separate numbers to interpret correctly under pressure at 3am.

replicationLagOk checks against a 60-second threshold

A nonzero replication lag is normal and expected; the threshold exists to catch the specific failure case where

replication has stalled entirely and the secondary region's data is meaningfully stale, which is exactly the condition that would make failing over right now unsafe.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener** for /admin/dr-readiness, then **AWS DynamoDB > Describe Table** for the Global Table.
- 2 Add **AWS CloudWatch > Get Metric Data** for the S3 replication latency metric.
- 3 Add an **HTTP Request** to the secondary region's health endpoint.
- 4 Add a **Transform Message** combining all three checks into one goForFailover verdict, then save and run, and add this endpoint's URL to the DR runbook document itself.

TESTING IN POSTMAN

GET https://localhost:8081/admin/dr-readiness

SAMPLE RESPONSE

JSON

```
{ "goForFailover": true, "replicaHealthy": true, "replicationLagSeconds": 4,
  "secondaryRegionUp": true }
```

BRIEFING ALL THE STEPS IN WORDS

The DynamoDB Connector's Describe Table operation confirms the Global Table's replica is active, the CloudWatch Connector's Get Metric Data operation checks that S3 replication lag is within a safe threshold, and an HTTP Request confirms the secondary-region worker is actually healthy, with a final transform combining all three into one clear go-or-no-go verdict — turning what would otherwise be three manual console checks under pressure into a single runbook step an on-call engineer can execute and trust in seconds.

#100

A Complete End-to-End MuleSoft-AWS Integration Platform: Ingestion, Processing, Storage, and Notification Tied Together

AWS

PROBLEM STATEMENT

This capstone brings the series full circle: an order arrives over HTTP, gets securely authenticated and validated, is durably archived and idempotently processed, triggers serverless enrichment and structured storage, notifies the customer and the warehouse, and reports its own health to a monitoring dashboard — one platform-grade flow demonstrating that everything built across five volumes composes into a single, production-ready integration.

DETAILED EXPLANATION

No single piece of this flow is new. What makes it a capstone is the composition: Volume 1's S3 archival, Volume 2's SQS/SNS messaging, Volume 3's Lambda and DynamoDB, Volume 4's KMS encryption and CloudWatch observability, and Volume 5's idempotency and orchestration patterns all appear here in their natural place in one coherent order-processing pipeline. This is deliberately the shape a real MuleSoft-AWS integration platform actually takes in production — not a single connector call, but a disciplined sequence of them, each one doing exactly the job its own problem in this series worked out in isolation.

CONNECTORS & COMPONENTS USED

CORE

HTTP Listener

AWS

AWS DynamoDB Connector – Put Item (idempotency claim)

AWS

AWS KMS Connector – Encrypt

AWS

AWS S3 Connector – Upload

AWS

AWS Lambda Connector – Invoke

AWS

AWS SQS / SNS Connectors – Publish

AWS

AWS SES Connector – Send Templated Email

AWS

AWS CloudWatch Logs / Connector – Audit & Metrics

ERR

Try / On Error Continue

STEPS TO CREATE THE MULE FLOW

- 1

Claim the order's idempotency key in DynamoDB, encrypt any PII field, and archive the order to S3.
- 2

Invoke a Lambda function to enrich and validate the order, then write the fulfillment state to DynamoDB.
- 3

Notify the warehouse via SQS, send the customer a confirmation via SES, and record success metrics and an audit log entry to CloudWatch.

ORDER-PLATFORM-FLOW



MULE CONFIGURATION XML

```

<flow name="order-platform-flow">
  <http:listener path="/platform/orders" config-ref="HTTP_Listener_config"/>
  <try>
    <dynamodb:put-item config-ref="DynamoDB_Config" doc:name="Claim idempotency key"
    tableName="processed-order-events"
      conditionExpression="attribute_not_exists(idempotencyKey)">
      <dynamodb:item>#{[ idempotencyKey: { S: payload.orderId } ]}</dynamodb:item>
    </dynamodb:put-item>

    <kms:encrypt config-ref="KMS_Config" doc:name="Encrypt customer PII"
      keyId="alias/order-platform-key" plaintext="#[payload.customerPhone]"
    target="phoneCipher"/>
    <s3:upload config-ref="S3_Config" doc:name="Archive order" bucketName="order-
    archive" key="#[payload.orderId ++ '.json']">
      <s3:content>#{write(payload - "customerPhone" ++ { phoneEncrypted:
    vars.phoneCipher.ciphertextBlob as String {encoding: "base64"} }, "application/json")}
    </s3:content>
    </s3:upload>

    <lambda:invoke config-ref="Lambda_Config" doc:name="Enrich and validate order"
      functionName="enrich-and-validate-order" invocationType="RequestResponse"
    target="enrichment"/>

    <dynamodb:put-item config-ref="DynamoDB_Config" doc:name="Record fulfillment state"
    tableName="order-fulfillment">
      <dynamodb:item>#{[ orderId: { S: payload.orderId }, status: { S: 'CONFIRMED' },
    confirmedAt: { S: now() as String } ]}</dynamodb:item>
    </dynamodb:put-item>

    <sqs:publish config-ref="SQS_Config" doc:name="Notify warehouse"
    queueUrl="${sqs.warehouseQueueUrl}"
      content="#[ orderId: payload.orderId ]"/>
    <ses:send-templated-email config-ref="SES_Config" doc:name="Confirm to customer"
      source="orders@example.com" templateName="order-confirmation"
      templateData="#[write({ orderId: payload.orderId }, "application/json")]>
    <ses:destination>
      <ses:to-addresses>#[payload.customerEmail]</ses:to-addresses>
    </ses:destination>
    </ses:send-templated-email>

    <cloudwatch:put-metric-data config-ref="CloudWatch_Config" doc:name="Record success
    metric" namespace="Integration/OrderPlatform">
      <cloudwatch:metric-data>#[[ { metricName: "OrdersProcessed", value: 1, unit:
    "Count" } ]]</cloudwatch:metric-data>
    </cloudwatch:put-metric-data>
  </try>
</flow>

```

```
<error-handler>
  <on-error-continue type="AWS_DYNAMODB:CONDITIONAL_CHECK_FAILED">
    <logger level="INFO" message="Duplicate order event – skipped"/>
  </on-error-continue>
</error-handler>
</try>
</flow>
```

EXPLANATION OF THE CODE

the entire pipeline wrapped in one Try scope, guarded by the idempotency claim at the very top

Placing the DynamoDB conditional claim as the first operation inside the Try scope means a duplicate order delivery is caught before any of the seven downstream steps — encryption, archival, enrichment, notification, or metrics — ever runs a second time for the same order.

```
payload - "customerPhone" ++ { phoneEncrypted: ... } before the S3 archive
```

This is the exact same field-substitution pattern from Problem #66, applied here to whichever field the order payload happens to carry as PII — the plaintext phone number is never present in the object that actually gets written to S3, consistent with everything Volume 4 established about protecting sensitive fields at rest.

STEPS TO BUILD IT IN ANYPOINT STUDIO

- 1 Drag an **HTTP Listener**, then wrap everything else in a single **Try** scope starting with the DynamoDB idempotency claim.
- 2 Add the KMS **Encrypt** and S3 **Upload** operations to protect and archive the order.
- 3 Add the Lambda **Invoke** operation for enrichment, then the DynamoDB **Put Item** operation recording fulfillment state.
- 4 Add the SQS **Publish**, SES **Send Templated Email**, and CloudWatch **Put Metric Data** operations, then an **On Error Continue** handler for the duplicate-key case, and save and run.

TESTING IN POSTMAN

POST `https://localhost:8081/platform/orders`

SAMPLE REQUEST BODY

JSON

```
{ "orderId": "ORD-9001", "customerEmail": "j.alvarez@example.com", "customerPhone": "555-0148", "sku": "SKU-4471", "quantity": 1 }
```

SAMPLE RESPONSE

JSON

```
{ "orderId": "ORD-9001", "status": "CONFIRMED", "archived": true, "notified": true }
```

BRIEFING ALL THE STEPS IN WORDS

An idempotency claim in DynamoDB guards the entire pipeline against duplicate delivery, KMS encryption and an S3 archive protect and durably store the order, a Lambda invocation enriches it and a DynamoDB write records its fulfillment state, and SQS, SES, and CloudWatch close the loop by notifying the warehouse, confirming with the customer, and recording the event for both metrics and audit — the same building blocks this entire series worked through one at a time, composed here into the shape a real MuleSoft–AWS integration platform actually takes in production.

End of Volume 5 — End of the Series

Volume 5 closed the series with the patterns that turn individually correct integrations into an enterprise-grade platform: multi-service orchestration, SES, Redshift, Glue, CloudFront, ECS/Fargate, and AWS Batch; idempotency, multi-region failover, DynamoDB Streams, event replay, the saga pattern for distributed transactions, cost-aware batching, blue-green deployment, monitoring feeds, and a disaster-recovery runbook — ending with a capstone flow tying the entire 100-problem series together into one order-processing platform.

Across all five volumes, this series worked through 100 real MuleSoft–AWS integration problems — S3 fundamentals, SQS/SNS/EventBridge messaging, Lambda/DynamoDB/Kinesis/Step Functions serverless and data patterns, security and resilience and observability, and finally advanced enterprise patterns — each with a problem statement, a detailed explanation, the connectors and components involved, the flow's design and canvas structure, syntax-highlighted Mule configuration XML, a plain-language walkthrough of that code, the steps to build it in Anypoint Studio, a Postman test, and a summary in words. Thank you for building through it.